

THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES,
LAHORE

Course Notes

Compiler Construction

CS-4031

Semester Fall-2026

Compiled lecture notes – see individual lectures for per-lecture references.

Contents

1	What Is a Language Processor?	3
2	Compilers vs. Interpreters vs. JIT Compilation	4
2.1	Pure Compilation (Ahead-of-Time)	4
2.2	Pure Interpretation	5
2.3	The Hybrid Reality: Bytecode + JIT	6
2.4	Comparing the Three Models	8
2.5	Case Study: How Java Achieves Portability	10
2.5.1	From Pure Interpreter to JIT: A Short History	10
2.5.2	Compilation to Bytecode, Then Portability via the JVM	10
2.5.3	HotSpot's Tiered JIT in More Depth	11
2.5.4	JIT vs. AOT vs. Interpreter: Advantages and Disadvantages	11
3	Why Study Compilers? (DB §1.1)	12
4	Applications of Compiler Technology (DB §1.5)	13
5	A Typical Language-Processing System	14
6	Different Kinds of Compilers	16
7	A Preview: What's Next	18
8	Self-Check Questions	18
1	The Phase Pipeline of a Compiler	20
1.1	Analysis and Synthesis: The Two Halves	21
2	Symbol Table and Error Handling: The Cross-Cutting Phases	21
3	Grouping the Phases: Front-End, Middle-End, Back-End	22
4	The Compiler's Neighbors: Preprocessor, Assembler, Linker & Loader	23
5	Self-Check Questions	26
1	Where Lexical Analysis Fits	28
1.1	Why Not Just One Phase?	29
2	Tokens, Patterns, and Lexemes	29
2.1	Worked Example 1: Tokenizing a Function Call	30
3	Attributes for Tokens	30
3.1	Worked Example 3: A Complete Statement Block	31
4	Lexical Errors	31
4.1	Error Recovery Strategies	32
5	Beyond the Dragon Book: How Real Lexers Handle Harder Cases	33
6	Self-Check Questions	35
1	Strings, Alphabets, and Languages	36
1.1	Operations on Languages	36

2	Regular Expressions	37
2.1	Worked Example: Building Up a Regular Expression	38
2.2	More Worked Examples	38
3	Regular Definitions	39
3.1	Worked Example: Identifiers	39
3.2	Worked Example: Unsigned Numbers (DB’s Classic Definition)	39
4	Extensions of Regular Expressions	39
5	Regular vs. Non-Regular Languages	40
6	Beyond the Dragon Book: Regex in Practice	41
7	Self-Check Questions	43

Lecture 1

How Programs Run: Compilers, Interpreters, and JIT Compilation (a Java Case Study); the Language-Processing System; Why Study Compilers; Kinds of Compilers

CLO(s) covered:	CLO 1
Dragon Book [DB]:	§1.1 (Language Processors), §1.2 (Language-Processing System – previewed here, detailed in Lecture 2), §1.4 (Compiler/Interpreter Distinction), §1.5 (Applications of Compiler Technology)
Other references:	[ET] Engineering a Compiler §1.1; [AP] Modern Compiler Implementation, Ch. 1

Here is what today actually covers:

Lecture Roadmap

1. What is a language processor, and what exactly is a *compiler*?
2. Compilers vs. interpreters vs. just-in-time (JIT) compilation — three ways to run a program, with a full case study of how **Java** uses this to become portable.
3. A **typical language-processing system**: preprocessor, compiler, assembler, linker, and loader – and exactly what an **object file** is.
4. Why study compilers at all, even if you never write one professionally?
5. Applications of compiler technology far beyond “turning C into an executable.”
6. Six different **kinds of compiler** you’ll encounter by name in industry.

This lecture is entirely conceptual — no algorithms yet. Lecture 2 goes much deeper into the internal *structure* (phases) of a compiler and the toolchain around it.

1 What Is a Language Processor?

[DB] A **language processor** is any program that takes another program written in some source notation and does something useful with it — runs it, translates it, checks it, or transforms it. Compilers are the most famous member of this family, but not the only one.

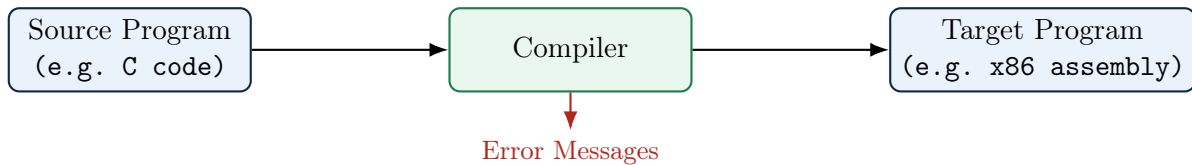
Definition: Compiler

A **compiler** is a program that reads a program written in one language — the *source language* — and translates it into an equivalent program in another language — the *target language*. Along the way, if the source program has errors, the compiler should report them to the user in a useful way.

Three things are packed into that definition and worth pulling apart, because exam questions love to test exactly this distinction:

- **Translation, not execution.** A compiler’s job is to *produce* a target program. It does not itself run your code. If you compile `main.c` with GCC, GCC’s job ends when it hands you `a.out`; a completely different program (the operating system’s loader, then the CPU) is what actually executes it.

- **Equivalence.** The target program must have the same meaning as the source program. This is the single hardest engineering promise a compiler makes, and essentially the entire field of compiler optimization exists to preserve this promise while still making the target program faster.
- **The target language need not be machine code.** Nothing in the definition says what the target language *is* – only that it is “another language.” Machine code is the single most common and most visible target, which is why people default to thinking “compiler = produces machine code,” but it is only one case. A compiler whose target is *another high-level language* is a source-to-source compiler (§6); a compiler whose target is bytecode for a virtual machine (`javac` → JVM bytecode, §2.3) is still a compiler by this same definition, even though no physical CPU can execute its output directly.



Exam Focus

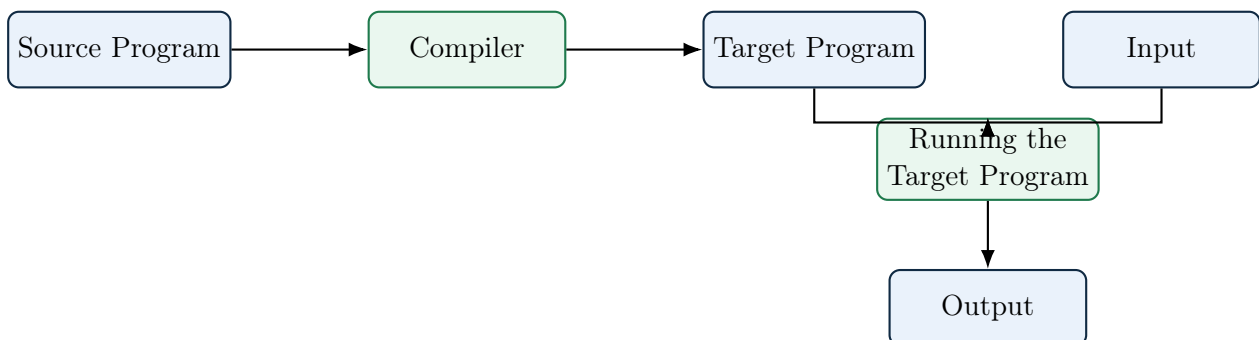
Be able to state, in one sentence, that a compiler *translates*; it does not *execute*. A one-line definition question like “define a compiler” is a classic first-quiz question and students routinely lose marks by describing what an interpreter does instead – or by adding “... into machine code” to the definition, which is not required and is contradicted by transpilers and by `javac` (§2.3).

2 Compilers vs. Interpreters vs. JIT Compilation

This is the heart of today’s lecture (DB §1.4). There are three broad strategies for getting from source code to a running program, and almost every real language-processing system you use day to day is one of these three, or (very commonly) a hybrid.

2.1 Pure Compilation (Ahead-of-Time)

[DB] The compiler translates the entire source program into a target program *before* execution begins, and the target program can then be run any number of times, independently of the compiler. In the classic AOT case that gives this strategy its name, the target language is machine code for the host machine – this is what C, C++, Rust, and (classic) Fortran compilers produce – but the “ahead-of-time, whole-program-at-once” strategy itself is independent of that choice of target (a transpiler, §6, is also AOT; it just targets another high-level language instead of machine code).



Advantage – maximal optimization opportunity. Because the entire source program is available before a single instruction runs, an AOT compiler can afford analyses and transformations that a system under run-time time pressure cannot: multiple, deliberately ordered optimization

passes; *whole-program* (interprocedural) analysis such as global register allocation or cross-function inlining; and expensive, exhaustive search over code-generation choices. None of this has to finish before the user sees output, since compilation happens entirely offline – only the resulting target program’s run time is on the user’s clock.

Disadvantage – the output is not portable. The target program an AOT compiler produces is machine code for one specific instruction set, calling convention, and (usually) operating system. A binary built for x86-64 Linux will not run on ARM64 macOS: there is no platform-independent artifact to ship. Reaching a new target means recompiling from source for that target (or cross-compiling, §6) – unlike an interpreter, which ships the same source to every platform, or a bytecode/JIT system, which ships the same portable bytecode everywhere.

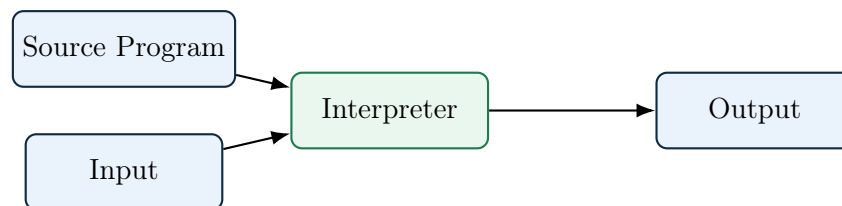
Trade-off, restated: compilation itself can be slow and happens “offline,” but every run of the target program afterward is fast, because all the translation work – and all the optimization it enables – has already been paid for once, at the cost of tying the result to one platform.

2.2 Pure Interpretation

[DB] An **interpreter** does not produce a separate target program at all. Instead, it directly executes the operations specified in the source program, using the source program (or a close representation of it) as input on every single run.

Definition: Interpreter

An interpreter is a language processor that reads a source program and, using the input supplied by the user, *directly* produces the output of that program — with no separate translation phase that produces standalone target code the user can run independently.



Classic (older) Python, shell scripts, and simple Basic interpreters are close to this pure model: the interpreter re-examines and re-dispatches on the source (or its parsed form) every single time a statement executes, even inside a loop that runs a million times.

Advantage – portability. The same source program (or bytecode) runs unmodified on any machine that has a compatible interpreter, with no separate build step per target platform. This is the mirror image of the AOT compiler’s disadvantage above.

Disadvantage – repeated-work overhead. There is no separate compile step, so you can start running – and get feedback on – code instantly, which is wonderful for interactive development and REPLs. The cost is that the interpreter re-examines and re-dispatches on the source (or its parsed form) on *every* execution of a line, with no persistent, whole-program analysis carried over between runs – the opposite of an AOT compiler’s one-time, multi-pass optimization. DB notes this repeated-work overhead can make interpretation **10–100× slower** than running compiled code.

Exam Focus

Know this speed trade-off in both directions: compilation trades slower “build time” for faster “run time”; interpretation trades away build time for slower, but more flexible and immediate, run time. Two further pairings are just as commonly tested: *portability* is a classic interpreter advantage over AOT compilation (the same source runs unmodified on any machine that has the interpreter, whereas an AOT binary is tied to one target and needs a recompile elsewhere), while *optimization opportunity* runs the other way – an AOT compiler sees the whole program before

run time and can afford multiple, expensive, whole-program passes, while a pure interpreter re-examines the source on every execution and so cannot afford to.

2.3 The Hybrid Reality: Bytecode + JIT

[beyond DB] In practice, almost no serious modern language is *purely* compiled or *purely* interpreted. DB §1.4 briefly mentions Java’s model as a hybrid; it is worth understanding this hybrid model properly, because it is how the overwhelming majority of code you run today — Java, C#, JavaScript, modern Python (via PyPy), and even Python’s own CPython — actually executes.

Definition: Virtual Machine (VM), vs. a Plain Interpreter

A **virtual machine**, in the sense used here, is a program that simulates a computer: it defines its own instruction set (an abstract, software-only ISA) together with the state a real CPU would have – a program counter, and either an operand stack (JVM, Wasm) or a set of virtual registers – and then runs a fetch-decode-execute loop over that instruction set, exactly the way real hardware executes machine code, except every step is simulated in software. Crucially, a VM does *not* interpret your source program directly. There is always a separate front-end compilation step first (described just below) that translates source into **bytecode** – instructions for this abstract machine – and the VM interprets *that*, not your original code.

This is different from a “plain” (tree-walking) interpreter, of the kind §2.2 first introduced: a tree-walking interpreter parses source into an AST and directly, recursively evaluates that tree (`evaluate(node)` calling `evaluate(node.left)`, and so on). There is no separate instruction set, no fixed opcode encoding, and no simulated CPU state – the evaluator’s own call stack mirrors the shape of the *source program*, not the shape of a machine. Nothing about it resembles hardware, so it does not earn the name “virtual machine.”

The distinction, side by side:

- **What is executed.** VM: a fixed, formally-defined bytecode instruction set, produced by a separate compile step. Tree-walking interpreter: the source program’s own parse tree, walked directly, with no separate instruction set at all.
- **Execution state.** VM: an explicit program counter plus an operand stack or virtual registers – state modeled on real hardware. Tree-walker: just the recursive call stack of the `evaluate` function, shaped like the program’s grammar, not like a CPU.
- **Is it a genuine compilation target?** VM: yes – bytecode is often documented and shared across multiple front-end languages (JVM bytecode is targeted by Java, Kotlin, Scala, and Clojure alike). Tree-walker: no – the AST is just this one interpreter’s private internal representation of this one program.

Every VM is (or contains) some form of interpreter – it interprets bytecode instruction by instruction. But not every interpreter is a VM: a tree-walker interprets source structure directly and never defines a machine to simulate. (A second, unrelated sense of “virtual machine” – *system* VMs like VMware or QEMU, which virtualize an entire computer including its OS – is a different concept entirely and not what is meant here.)

1. **Source → bytecode.** A “front-end” compiler translates source code into a compact, machine-independent intermediate form called **bytecode** (Java’s `.class` files hold JVM bytecode; Python’s `.pyc` files hold Python bytecode).
2. **Bytecode → execution.** A **virtual machine (VM)** then runs this bytecode. It can do this two ways:
 - *Interpret* the bytecode directly, instruction by instruction (slow but starts instantly) – this is what most bytecode VMs do at first.
 - *Just-in-time (JIT) compile* hot pieces of bytecode — the loops and functions that run

over and over — into native machine code *while the program is running*, and cache that native code for reuse.

Definition: Just-In-Time (JIT) Compilation

JIT compilation defers translation to native machine code until run time, and does so selectively: only code that is actually hot (executed frequently) is compiled, while cold code may remain interpreted. This lets a system get the fast startup of an interpreter and the fast steady-state speed of a compiler.

Why “Warm-Up” Adds a Delay

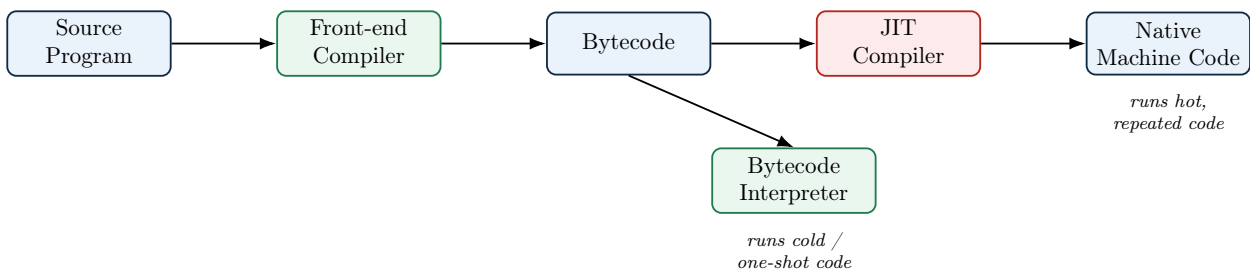
The “time to reach peak speed” row in §2.4’s table (distinct from startup latency – see the exam-focus note there) has a concrete cause: three costs stack up, in sequence, before any piece of code reaches its fastest version, and none of them can be skipped.

1. **Early runs pay interpreter-speed cost.** The very first time a method or loop executes, no compiled native code for it exists yet – there is nothing to jump to – so it runs through the interpreter (or a cheap baseline tier, e.g. HotSpot’s C1), which is inherently slower per operation than optimized native code.
2. **The system must first observe that code is hot.** A JIT does not compile on the first call; it counts. The runtime increments an invocation/back-edge counter on every call or loop iteration and only triggers compilation once that counter crosses a threshold. A loop that only runs a handful of times may never get compiled at all, because counting to that threshold, by construction, requires real executions to have already happened at the slower tier.
3. **Compilation itself takes real, non-trivial time.** Once flagged hot, the method still has to go through the JIT’s own pipeline – build an IR from the bytecode, run optimization passes (inlining, register allocation, and more), and emit native code. This usually runs on a background thread so it does not block the program outright, but it still competes for CPU and memory (see “Extra runtime overhead” in §2.4’s table), and the *old*, slower version keeps running until the newly compiled version is ready and swapped in.

The practical consequence: long-running processes (servers, long batch jobs) amortize this one-time warm-up cost over a long steady-state run and benefit enormously from JIT compilation, while short-lived scripts or CLI invocations may finish before ever reaching peak speed – exactly why AOT-vs.-JIT benchmarks can be misleading if they measure only a program that runs for a second or two.

Example: Tiered execution in real systems

- **JVM (HotSpot):** interprets bytecode first; the C1 (“client”) JIT compiles warm methods quickly with light optimization; the C2 (“server”) JIT recompiles the hottest methods with heavy optimization once enough profiling data exists.
- **V8 (Chrome/Node.js JavaScript engine):** Ignition interprets bytecode; Sparkplug does a fast baseline JIT; TurboFan does an aggressive optimizing JIT for hot functions, and can *deoptimize* back to the interpreter if an assumption used for optimization (e.g. “this variable is always a number”) turns out to be wrong.
- **CPython (standard Python):** traditionally a pure bytecode interpreter with no JIT; PyPy is an alternative Python implementation that adds a JIT and is often 5–10× faster on long-running code as a result.



Extra Depth: AOT with a JIT Fallback

DB draws the compiler/interpreter line fairly simply. In practice, the line is blurrier still: some modern systems (GraalVM Native Image, .NET's ReadyToRun) precompile most code AOT for fast startup but still keep a JIT available for code discovered or specialized at run time. So “compiled” and “JIT-compiled” are not always mutually exclusive labels for one system. *(Transpilers, cross-compilers, bootstrapping compilers, and decompilers are their own category of “kind of compiler” rather than a compiler/interpreter distinction – we cover all of these properly later in this lecture.)*

2.4 Comparing the Three Models

Table 1: Comparing the three execution models

	Pure Compiler	Pure Interpreter	JIT / Hybrid
When translation happens	Entirely before run time	Continuously, during run time	Source → bytecode happens before run time; bytecode → native code (the JIT step itself) is always fully during run time – see note below
Time to reach peak speed	N/A – already at peak from instruction one (see above)	N/A – there is no “peak”; every instruction runs at the same interpreted speed forever	Nonzero (“warm-up”) – profiling must first identify hot code, then compilation must finish, before that code is upgraded to native speed; the program is already running the whole time this happens
Steady-state speed	Fastest	Slowest (re-dispatch overhead)	Approaches compiled speed for hot code
Portability	Target is machine-specific – a new platform needs a recompile (or cross-compile)	Source runs anywhere the interpreter exists	Bytecode is portable; the JIT-generated native code is not, but it never has to be shipped
Optimization opportunity	Largest – whole program visible before run time, so multiple, expensive, whole-program passes are affordable	Smallest – source (or bytecode) is re-examined on every execution, so heavy-weight analysis is normally too costly to repeat	Bounded by run-time budget – fewer/cheaper passes than AOT and limited mostly to hot methods, but compensated by real profiling data (next row) unavailable to AOT

continued on next page

Table 1 – comparing the three execution models (continued)

	Pure Compiler	Pure Interpreter	JIT / Hybrid
Can use run-time profiling info?	No – only sees the source, never the running program	N/A – re-examines source every time, no persistent profile	Yes – observes actual branch frequencies and value types, enabling speculative optimizations no static compiler can safely make
Extra runtime overhead	None – compiler is long gone by run time	Dispatch overhead only	Profiling counters, background compiler threads, and a generated-code cache all compete with the program for CPU/memory
Debuggability / interactivity	Weakest (must recompile to test a change)	Best (REPL, immediate feedback)	Good (often still supports a REPL over bytecode)
Typical users	C, C++, Rust, Fortran	Shell scripts, classic BASIC	Java, C#, JavaScript, PyPy

Exam Focus

The “When translation happens” row is the row most often misread. The hybrid pipeline has *two* separate translation stages, and only one of them is “just-in-time”:

- **Source** → **bytecode** (e.g. `javac` producing `.class` files) is ordinary ahead-of-time compilation. It happens once, before the program is ever run, exactly like §2.1’s pure-compiler case – it is just that its *target* language is bytecode instead of machine code.
- **Bytecode** → **native code**, the actual JIT step, is *never* done ahead of time, not even partially. The decision of *which* loop or method is hot enough to be worth compiling is made by watching the program actually run (via profiling counters), and the native code generated is specific to the behavior observed *during that run* – this is exactly what “just-in-time” means: translation deferred until the last possible moment, triggered by run-time evidence, not decided in advance.

So the “partly before, partly during” language in the table describes the *full hybrid pipeline* (front-end compile, then JIT), not the JIT step in isolation. Taken alone, JIT compilation is entirely a run-time activity, no different in *when* it happens from pure interpretation – what makes it different is *what* it produces (a reusable native-code artifact) rather than *when* it produces it.

Exam Focus

A related trap, worth knowing even though the table above does not track it separately: “startup latency” and “warm-up delay” sound like the same thing but are not. **Startup latency** asks only: how long before the program starts executing at all? By that measure, JIT and pure interpretation are identical – a JIT-based system *begins* by interpreting (or running a cheap baseline tier) exactly like a pure interpreter does, so there is no extra pause before the first instruction runs. “Warm-up,” by contrast, is not a delay *before* starting; it is the time *after* the program has already started before some of its code gets promoted to native speed – exactly what the “Time to reach peak speed” row above captures. The program is running the entire time warm-up is happening – it is simply running at interpreted (not yet peak) speed for that stretch. So it is correct to say a JIT system has low startup latency *and* a nonzero warm-up period – these are not in tension, because they answer two different questions.

A Bit of History

Interpretation is not a modern shortcut invented for scripting languages — it is *older* than compilation. Early LISP (1958) was interpreted because writing a LISP compiler was itself a research problem; interpreters let you experiment with a language before anyone knew how to compile it well. JIT compilation is also older than most people assume: the term traces to Smalltalk and self-modifying-code systems of the 1980s, and was popularized for the mainstream by Sun’s HotSpot JVM in 1999 – it was Java’s answer to “we want C-like speed but Smalltalk-like portability and safety.”

2.5 Case Study: How Java Achieves Portability

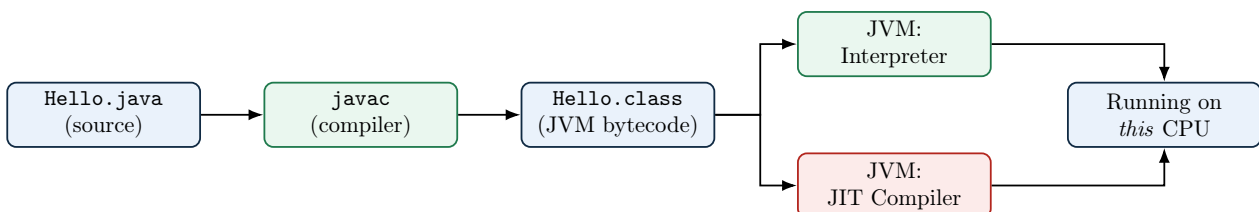
[beyond DB] Java is the single clearest real-world illustration of everything in §2, so it is worth walking through as a complete case study — especially since Java’s own design goals were *explicitly* about the compiler/interpreter trade-off from the start.

2.5.1 From Pure Interpreter to JIT: A Short History

Era	What Happened
Java 1.0 (1996)	The original JVM was a pure bytecode interpreter . Every byte-code instruction was fetched, decoded, and dispatched on every single execution – simple and portable, but roughly 10–20× slower than equivalent compiled C, which badly hurt Java’s early reputation for performance.
Java 1.2 – “HotSpot” research	Sun’s research team built an adaptive compiler codenamed HotSpot : instead of compiling everything, or nothing, it would <i>watch the program run</i> and compile only the small fraction of methods that actually matter for performance.
Java 1.3 (2000) onward	HotSpot became the default production JVM. This is the point Java became a genuine hybrid : start by interpreting (fast startup), then JIT-compile whichever methods turn out to be “hot” (fast steady state). Every mainstream JVM since has kept this same architecture.

2.5.2 Compilation to Bytecode, Then Portability via the JVM

[DB] `javac`, the Java *compiler*, does not produce machine code for any real CPU at all. It produces **Java bytecode** – instructions for an abstract, idealized machine called the **Java Virtual Machine (JVM)** that does not physically exist as hardware.



“Write Once, Run Anywhere”

Because `javac` only ever has to target *one* abstract machine (the JVM specification) instead of every physical CPU family, the exact same `Hello.class` file runs unmodified on Windows/x86, Linux/ARM, or a JVM on any other platform — as long as *that* platform has a compliant JVM installed. All the platform-specific work is pushed into the JVM implementation itself (one per platform, built once by the JVM vendor), instead of into every single Java program ever written. This is Sun Microsystems’ famous “Write Once, Run Anywhere” pitch for Java, and it is the exact same $M + N$ argument (rather than $M \times N$) you will meet again with LLVM IR in Lecture 2 — just applied to distribution instead of to compiler internals.

2.5.3 HotSpot’s Tiered JIT in More Depth

[DB] Modern HotSpot uses **tiered compilation**, layering the interpreter and two JIT compilers so the JVM is never stuck doing more work than the code’s importance justifies:

1. Every method starts out **interpreted**, while the JVM silently maintains an **invocation counter** (calls) and a **back-edge counter** (loop iterations) for it.
2. Once a counter crosses a threshold, the **C1 (client) compiler** quickly produces a lightly-optimized native version, and also inserts **profiling instrumentation** into it (recording which branches are taken, which concrete types show up at a call site, ...).
3. If the method keeps getting hotter, the **C2 (server) compiler** recompiles it a second time, now using the profiling data gathered by the C1 version to make aggressive, *speculative* optimizations: inlining a virtual method call that has – so far – always resolved to the same concrete type; eliminating a heap allocation entirely if it can prove the object never escapes the method (**escape analysis**); unrolling a hot loop.
4. If a speculative assumption is later violated (a second, different type shows up at that call site), HotSpot **deoptimizes**: it discards the C2 code for that method and falls back to the interpreter (or C1) until it is confident enough to try recompiling again.

Exam Focus

Be able to explain, in your own words, why HotSpot’s C2 compiler can safely make optimizations (like assuming a call site’s type never changes) that a purely static AOT compiler like GCC never could – and what mechanism (deoptimization) lets it recover if that assumption turns out to be wrong.

2.5.4 JIT vs. AOT vs. Interpreter: Advantages and Disadvantages

Pulling together everything from the enhanced table in §2.4 and the HotSpot walkthrough above, here is the direct trade-off summary for a JIT compiler specifically:

JIT Advantages

Can use **real runtime profile data** (actual types, actual branch frequencies) to make optimizations no static compiler can safely attempt – generally impossible for an AOT compiler, which only ever sees the source.

Ships one **portable** bytecode artifact instead of a separate binary per target CPU – matching an interpreter’s portability, unlike AOT.

Reaches **steady-state speed close to AOT-compiled native code** for long-running, hot-path-dominated programs – far faster than a pure interpreter ever gets.

Can **adapt to the actual host CPU** at run time (e.g. use available vector instructions) without shipping separate binaries per microarchitecture, unlike a single AOT build.

JIT Disadvantages

Warm-up latency: peak performance is only reached after enough executions to get hot code recompiled – a real problem for short-lived programs (e.g. CLI tools, serverless functions).

Consumes **extra CPU and memory at run time** for profiling counters, the JIT compiler threads themselves, and a generated-code cache – resources an AOT binary or a plain interpreter never spend.

Less predictable performance moment-to-moment (a method might pause execution while it gets recompiled), which matters for hard real-time systems that AOT compilation suits better.

Significantly more implementation complexity: an interpreter, multiple JIT tiers, profiling instrumentation, and deoptimization machinery, versus AOT’s comparatively simple “compile once, done.”

Extra Depth: The Java Class File, Briefly

A `.class` file is itself a small, well-defined binary format worth knowing at a glance: it opens with a **magic number** (`0xCAFEBAFE`, a Sun Microsystems in-joke), a class-file version number, a **constant pool** (a table holding every literal, class name, and method/field reference the class uses – similar in spirit to the symbol table we will meet in Lecture 2), and then the actual bytecode for each method. Tools like `javap -c Hello.class` let you disassemble a compiled class and read its raw JVM bytecode instructions directly – a nice hands-on way to see §2’s “token stream → bytecode” pipeline made completely concrete.

3 Why Study Compilers? (DB §1.1)

[DB] You may never write a production C compiler. So why is this course core, and why does almost every strong CS program require it? Three overlapping reasons:

1. **Compilers are, in DB’s phrase, “one of the largest and most complex software systems” commonly built** – yet a working one is buildable in a single semester project. Nowhere else in the curriculum will you build something this large, this precisely specified, end to end, from a formal problem statement to a working artifact.
2. **The techniques generalize far beyond compilers.** Finite automata power text editors, network protocol parsers, and input validation. Context-free grammars appear in JSON parsers, config-file readers, and query languages. Data-flow analysis underlies static analyzers, IDE “find usages” features, and security vulnerability scanners. You are not just learning to build a compiler; you are learning the standard toolkit for building *any* system that must process structured, language-like input reliably.
3. **Compilers sit at the hardware/software boundary.** Understanding how a `for` loop becomes register moves and branch instructions demystifies performance – why one piece of “equivalent” code runs 10× faster than another, why cache-friendly code matters, and why some

languages can be faster than others for reasons that have nothing to do with the programmer's skill.

Extra Depth: A “Meta” Argument

There's a fourth reason DB does not spell out explicitly but that experienced engineers often cite: compiler construction is one of the few courses where you are forced to build a *multi-stage pipeline* with cleanly separated concerns (lexing, parsing, semantic analysis, code generation) and *formally specified* correctness at each stage. This is directly transferable to designing any large pipeline system – data pipelines, network stacks, even ML training pipelines – where the same discipline of “define an intermediate representation, verify it, transform it, repeat” shows up again and again.

4 Applications of Compiler Technology (DB §1.5)

[DB] DB is explicit that compiler techniques reach “far beyond” translating a general-purpose programming language into machine code. Five application areas DB highlights:

1. **Implementation of high-level programming languages.** The obvious case, but note the historical point DB makes: higher-level languages raised the level of abstraction precisely *because* compiler technology matured enough to make that abstraction affordable in performance terms.
2. **Optimizations for computer architectures.** As hardware has grown more parallel (multi-core, SIMD/vector units, deep pipelines, GPUs), the compiler has taken on more responsibility for finding parallelism and hiding memory latency that programmers cannot realistically manage by hand.
3. **Design of new computer architectures.** Hardware designers now co-design instruction sets *with* the compiler in mind (RISC philosophy: keep hardware simple, let the compiler do the scheduling), since a feature only helps if a compiler can reliably exploit it.
4. **Program translations.** Beyond source-to-machine-code, DB lists: binary translation (re-targeting compiled binaries to a new instruction set, e.g. Apple's Rosetta 2 translating x86 binaries to run on Apple Silicon), hardware synthesis (compiling a hardware description language like Verilog into actual circuit layouts), database query interpreters (translating SQL into an execution plan), and compiled simulation (compiling a simulation model, e.g. for chip verification, into fast executable code rather than interpreting it).
5. **Software productivity tools.** Type checking, bounds checking, and static program-analysis tools that catch bugs before run time all reuse the same front-end machinery (lexers, parsers, semantic analyzers) that compilers use.

Extra Depth: Applications DB Does Not Mention

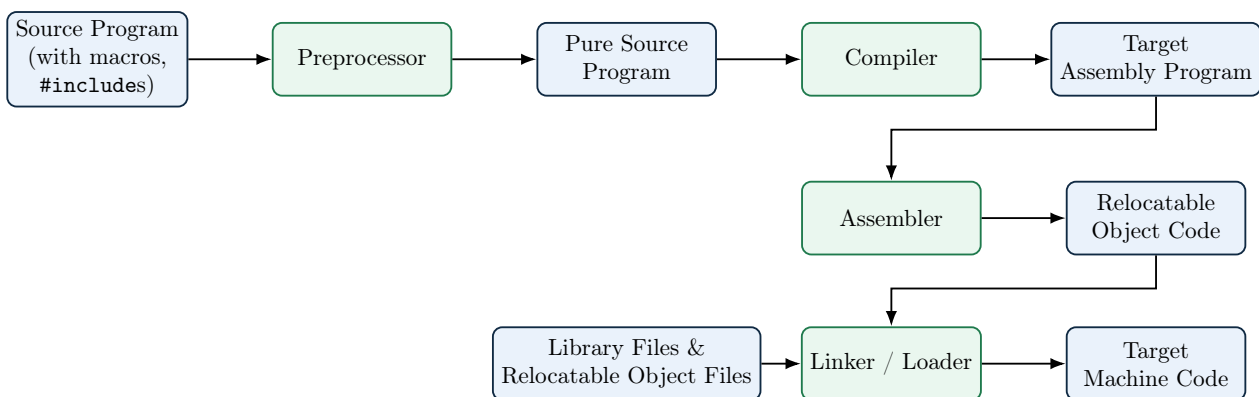
The five areas above were written with early-2000s computing in mind. Several major compiler-technology application areas have grown enormously since and deserve a mention:

- **Machine-learning compilers.** Frameworks like Google's XLA, Apache TVM, and MLIR (Multi-Level Intermediate Representation, itself built using classic compiler theory) compile neural-network computation graphs down to highly optimized GPU/TPU kernels – this is now one of the most active areas of compiler research.
- **WebAssembly (WASM).** A portable, sandboxed bytecode target designed so *any* source language (C, Rust, Go, ...) can compile to it and run safely inside a browser or standalone runtime at near-native speed.

- **Superoptimizers.** Tools (e.g. Souper, STOE) that search – sometimes with SMT solvers, sometimes with reinforcement learning – for provably optimal short instruction sequences, going beyond what traditional rule-based optimization passes can find.
- **Domain-specific languages (DSLs) and their compilers.** Halide (image-processing pipelines), SQL query planners, regular-expression engines, and shader languages (GLSL/HLSL for GPUs) are all compilers for a language deliberately kept small and specialized so it can be optimized far more aggressively than a general-purpose language could be.
- **Security applications.** Control-flow integrity instrumentation, automatic bounds-check insertion (AddressSanitizer), and fuzzing harness generation are compiler-pass techniques applied specifically to hardening software against attack.
- **LLVM as shared infrastructure.** Perhaps the biggest structural change since DB 2nd edition: LLVM lets dozens of front-ends (Clang/C/C++, Rust, Swift, Julia, and more) share one very well-engineered optimizer and set of back-ends, instead of every language reinventing code generation. We will return to LLVM in Lecture 25.

5 A Typical Language-Processing System

[DB] The “compiler” most people invoke by typing `gcc file.c` is not, strictly, one single program at all – it is a **driver** that coordinates several distinct tools in sequence. DB places the compiler proper inside this larger **language-processing system**.



A typical language-processing system (compare DB Fig. 1.5): the “compiler” you invoke on the command line is really a pipeline of five distinct programs.

- **Preprocessor.** Expands macros (`#define`), includes files (`#include`), and resolves conditional compilation directives (`#ifdef`) — all as pure text substitution, before the compiler’s grammar-aware phases ever run.
- **Compiler (proper).** Everything from §1: source $\rightarrow$ target assembly (today’s “black box”; Lecture 2 opens it up phase by phase).
- **Assembler.** Translates human-readable assembly instructions into **relocatable machine code** and packages it as an **object file**.
- **Linker.** Combines multiple object files and library files into one program, resolving references between them.
- **Loader.** Places the final program into memory and starts it running.

Is an Assembler a Compiler?

By the definition in §1, yes: an assembler reads a program in one language (assembly) and translates it into an equivalent program in another language (relocatable machine code), with no execution of its own along the way – that is exactly the compiler definition. It is given its own name and its own box in the pipeline diagram above for practical, not definitional, reasons: assembly is designed to be an almost line-for-line, one-to-one notation for machine instructions, so an assembler’s job skips most of what makes a “full” compiler hard (there is essentially no analysis phase, no optimization search, and little structural gap between source and target to bridge) and reduces mostly to table lookup and address bookkeeping. The lesson generalizes: *compiler* names a *role* – translate one language into another, without executing either – not a claim about how sophisticated the translation has to be. An assembler, a JVM-bytecode compiler like `javac`, and a heavily-optimizing tool like GCC are all compilers in exactly the same technical sense; they differ only in how much analysis and transformation the translation requires.

Definition: Object File

An **object file** is the output of the assembler: a binary file containing machine instructions and data *that is not yet a runnable program*. Critically, it contains:

- **Relocatable code/data**, with memory addresses left as placeholders, since the assembler does not know where in memory this code will ultimately live;
- A table of **symbols this file defines** (e.g. a function it implements, available for other files to call); and
- A table of **symbols this file references but does not define** (e.g. a call to `printf`, defined elsewhere in the C standard library) – these are exactly what the linker must resolve next.

On Linux this is the `.o` file produced by `gcc -c`; the same idea is called an `.obj` file on Windows.

What the Linker Actually Does

Programs are almost never compiled as a single file. The **linker**’s job is to take *several* object files (and pre-built library files) and merge them into one program, by:

1. **Symbol resolution**: matching every symbol a file *references* (like a call to a function defined in another file, or in a library) to the file that actually *defines* it. An unresolved symbol here is exactly the “undefined reference to `foo`” error you see at link time, as distinct from a compiler error.
2. **Relocation**: now that every piece of code has been assigned a real, final address in the combined program, the linker patches every placeholder address left by the assembler with the correct final value.

The output is one combined file — an **executable** — with (in the simplest case) no unresolved symbols and no remaining placeholder addresses.

What the Loader Actually Does

The **loader** is typically part of the operating system, not the compiler toolchain. When you run a program, the loader:

1. Allocates memory for the program (code, data, stack, heap regions);

2. Copies (or memory-maps) the executable's instructions and initial data into that memory;
3. Performs any *final* address fix-ups that were deferred until this exact moment (relevant for dynamically linked libraries, previewed in Lecture 2); and
4. Transfers control to the program's entry point – the actual first instruction that runs, which is *not* always your `main` function directly (a small language-runtime startup routine usually runs first).

Exam Focus

A very common exam trap: “which phase reports an ‘undefined reference’ / ‘unresolved external symbol’ error for a missing function?” The answer is the **linker**, not the compiler — the compiler is perfectly happy as long as a function is *declared* (e.g. via a header file); only the linker checks that a matching *definition* actually exists somewhere among all the object files and libraries it was given.

A Preview of What's Coming in Lecture 2

This is deliberately the *introductory* pass over the toolchain. Lecture 2 goes much further: static vs. dynamic linking, position-independent code and ASLR, real object-file formats (ELF, Mach-O, PE), and how build systems like Make, Ninja, and Bazel orchestrate this whole pipeline across thousands of files.

6 Different Kinds of Compilers

[beyond DB] “Compiler” is used as an umbrella term across the industry for several genuinely different tools, distinguished by *what kind of language* sits on each side of the translation (high-level vs. low-level) and *which machine* runs the result. Six worth knowing by name:

Kind	Source $\rightarrow$ Target	Example(s)
Native (AOT) compiler	High-level $\rightarrow$ machine code, same target as the host machine	GCC, Clang, <code>rustc</code> compiling for the machine they run on
Source-to-source compiler (transpiler)	High-level $\rightarrow$ a <i>different</i> high-level language	TypeScript $\rightarrow$ JavaScript; Babel (modern JS $\rightarrow$ older JS); CoffeeScript $\rightarrow$ JavaScript
Cross-compiler	High-level $\rightarrow$ machine code for a target <i>different</i> from the host running the compiler	Compiling ARM binaries for a phone on an x86 laptop; embedded firmware toolchains
Binary translator	Low-level (one instruction set) $\rightarrow$ low-level (a <i>different</i> instruction set)	Apple's Rosetta 2 (x86-64 $\rightarrow$ ARM64); QEMU's user-mode emulation
Decompiler	Low-level (machine code or bytecode) $\rightarrow$ high-level- <i>like</i> source	Ghidra, IDA Pro (machine code); JD-GUI, CFR (Java bytecode)
Bootstrapping (self-hosting) compiler	A compiler for language X, <i>written in X</i> itself	GCC (mostly C); <code>rustc</code> (written in Rust)

Example: Two of These, Worked Through

Binary translation (Rosetta 2): when Apple moved Macs from Intel (x86-64) to Apple Silicon (ARM64) in 2020, millions of existing apps were compiled x86-64 binaries with no source available to the end user. Rosetta 2 translates those already-compiled x86-64 binaries into ARM64 machine code – notice both sides of this translation are *low-level*, which is what makes it a binary translator rather than an ordinary compiler.

Bootstrapping (GCC): you cannot compile the very first C compiler *using* a C compiler, since none exists yet. Historically, an initial, minimal C compiler was written in assembly or an even earlier language; once that minimal compiler could compile *a subset* of C, a fuller C compiler was written in C and compiled by the minimal one; that fuller compiler then compiles *itself* (and every subsequent version) from then on. This chicken-and-egg-breaking process is exactly what “self-hosting” means.

Exam Focus

Given a described tool (“translates Kotlin to Java source,” “translates ARM machine code back into readable pseudo-C,” “compiles Swift for an iPhone from a Mac”), be able to name which of these six kinds it is and justify your answer using the *source $\rightarrow$ target* column above – this classification-by-example is a very common short-answer format.

Where the Compiler-vs-Interpreter and Kind-of-Compiler Axes Meet

These two classification schemes are independent and can combine freely. A JIT compiler *is* a native compiler by this section’s scheme (high-level/bytecode $\rightarrow$ machine code), just one whose *timing* (during execution) differs from an AOT native compiler’s. A transpiler like Babel is almost always itself an AOT tool – it finishes its work entirely before the JavaScript it produces ever runs. Don’t confuse “what kind of translation is this” (this section) with “when does the translation happen” (§2) – exam questions sometimes test both axes on the same tool deliberately.

7 A Preview: What’s Next

Lecture 2 opens up the compiler “black box” from §1 of this lecture and walks through its internal phases – lexical analysis, parsing, semantic analysis, intermediate code generation, optimization, and final code generation – in real depth. It also returns to the language-processing toolchain from §6, going far beyond today’s introduction: static vs. dynamic linking, position-independent code, real object-file formats, and how build systems orchestrate compilation across large projects. Everything in today’s lecture is about *what* these systems do from the outside; Lecture 2 begins answering *how*, in detail.

8 Self-Check Questions

Practice – Not Graded

1. In one sentence, explain why “a compiler executes your program” is a factually incorrect statement, and what does execute it instead.
2. Classic CPython has no JIT; PyPy does. Predict, with a reason, which one would be faster for a program consisting of a single loop that runs 50 million times, versus a script that runs once and exits immediately.
3. Explain, using the $M + N$ vs. $M \times N$ argument, why compiling to JVM bytecode makes Java portable in a way that compiling directly to x86 machine code would not.
4. A program compiles successfully with no errors, but fails at link time with “undefined reference to `compute_total`.” Which tool caught this, and why couldn’t the compiler have caught it earlier?
5. Is a transpiler (e.g. TypeScript $\rightarrow$ JavaScript) a compiler under DB’s definition? Justify your answer using the definition given in §1, and separately, name which of the six “kinds of compiler” from §6 it belongs to.
6. Give one example, not listed in this lecture, of a real tool on your own laptop that is best described as an “interpreter” rather than a compiler.
7. HotSpot’s C2 JIT only compiles a method after it has been *interpreted* many times. Why not just JIT-compile every method the very first time it’s called?

Key Takeaways

- A **compiler translates**; it does not itself execute the source program. An **interpreter** executes the source directly, with no independent target program left behind.
- Compilers trade slower build time for faster, repeatable run time, at the cost of a target

binary that is **not portable** (tied to one ISA/OS) and paid for by the whole-program visibility that enables aggressive, multi-pass optimization. Interpreters trade that away for instant startup, easy interactivity, and **source-level portability**, at the cost of repeated re-interpretation overhead on every run.

- Most real systems today (JVM, V8, .NET, PyPy) are **hybrids**: compile to portable byte-code, interpret it at first, then **JIT-compile** the hot paths to native code at run time – Java’s own history (pure interpreter in 1.0, HotSpot’s tiered JIT from 1.3 onward) is the clearest real example, and it’s what makes “write once, run anywhere” work.
- A JIT’s big edge over AOT is **using real runtime profile data** for speculative optimization; its costs are **warm-up latency** and **runtime profiling/compilation overhead** – know this trade-off table cold.
- The compiler proper sits inside a larger **language-processing system**: preprocessor → compiler → assembler → linker → loader. The assembler produces an **object file** (relocatable code plus defined/referenced symbol tables); the **linker** resolves symbols and relocates addresses; the **loader** places the program in memory and starts it.
- “Compiler” covers several genuinely different tools: **native/AOT compilers**, **source-to-source transpilers**, **cross-compilers**, **binary translators**, **decompilers**, and **bootstrapping/self-hosting compilers** – know the source→target shape of each.
- Compiler techniques (automata, grammars, data-flow analysis) show up far outside traditional compilers – in IDEs, static analyzers, database engines, and ML frameworks – which is the real reason this course is core, not elective.
- DB §1.5’s five application areas (language implementation, architecture optimization, architecture design, program translation, productivity tools) have since been joined by ML compilation, WebAssembly, and shared infrastructure like LLVM.

References for This Lecture

- DB Aho, Lam, Sethi, Ullman. *Compilers: Principles, Techniques, and Tools*, 2nd ed., §1.1, §1.2, §1.4, §1.5.
- ET Cooper & Torczon. *Engineering a Compiler*, 2nd ed., §1.1.
- AP Appel. *Modern Compiler Implementation in C/Java/ML*, Ch. 1.

Lecture 2

Structure of a Compiler: Phases Overview; Front-/Middle-/Back-End; Preprocessor, Assembler, Linker, Loader

CLO(s) covered:	CLO 1
Dragon Book [DB]:	§1.2 (The Structure of a Compiler), §2.1–§2.8 (overview of a syntax-directed translator's stages)
Other references:	[PP] Programming Language Pragmatics §1.5, §1.6

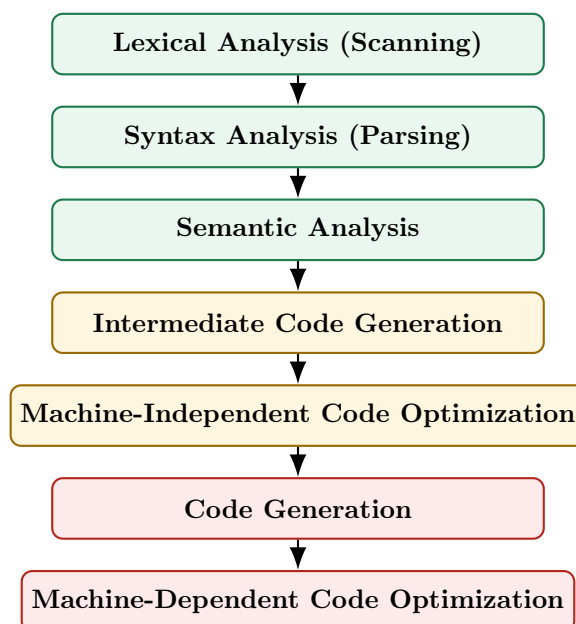
Lecture Roadmap

1. The classical phase pipeline of a compiler (DB Fig. 1.6): what each phase consumes and produces.
2. The two cross-cutting components every phase touches: the **symbol table** and **error handling**.
3. Grouping the phases into **front-end** / **middle-end** / **back-end**, and why that grouping is an engineering decision, not just a diagram convenience.
4. The compiler's **neighbors** in the full toolchain: preprocessor, assembler, linker, and loader — what runs before and after the compiler proper.

Today is still “the outside view.” From Lecture 3 onward we go phase-by-phase in depth, starting with lexical analysis.

1 The Phase Pipeline of a Compiler

[DB] DB organizes a compiler as a pipeline of phases, each of which takes the output of the previous phase as input, checks it, and produces a new representation of the program one step closer to executable code.



Phase	Input	Output
Lexical Analysis	Raw character stream	Stream of tokens (e.g. ID, NUM, +)
Syntax Analysis	Token stream	Syntax tree (or parse tree) capturing grammatical structure
Semantic Analysis	Syntax tree	Annotated (“decorated”) tree: types checked, scopes resolved
Intermediate Code Gen.	Annotated tree	Machine-independent IR , e.g. three-address code
Code Optimization	IR	Improved, semantically-equivalent IR
Code Generation	Optimized IR	Target-machine code (often assembly)

Exam Focus

Memorize the phase order and, for each phase, one sentence describing its input and output. This exact table (input → phase → output) is the single most common short-answer / MCQ pattern in compiler exams worldwide, not just here.

1.1 Analysis and Synthesis: The Two Halves

[DB] DB groups these six phases into two conceptual halves:

- **Analysis part (“front end”)**: lexical, syntax, and semantic analysis, plus intermediate code generation. This half *breaks down* the source program and builds an intermediate representation plus a record of the program’s structure (the symbol table). Analysis is where most **compile-time errors** are caught and reported.
- **Synthesis part (“back end”)**: optimization and code generation. This half *constructs* the desired target program from the intermediate representation and symbol table.

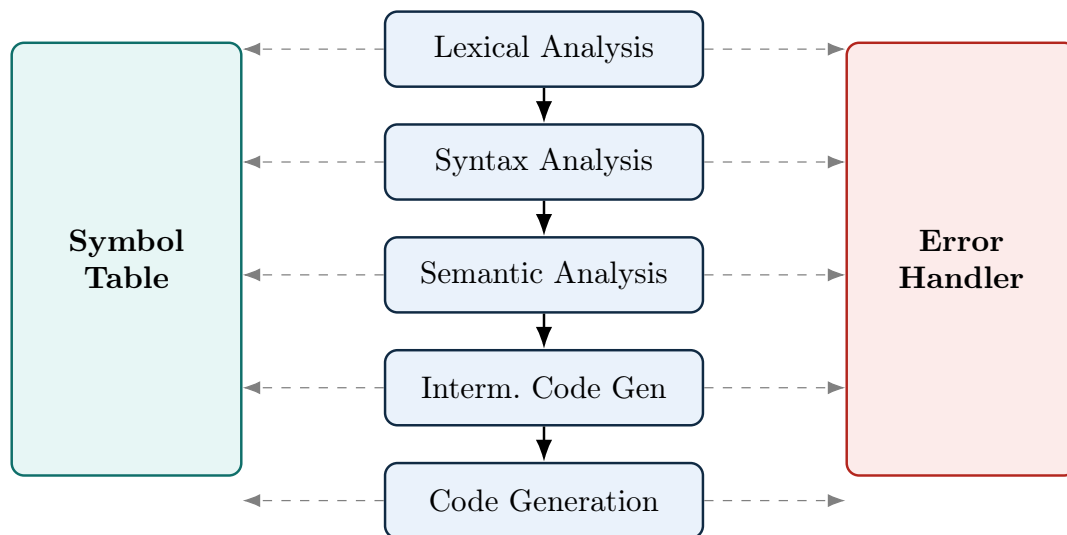
Definition: Analysis-Synthesis Model

A compiler can be understood as consisting of an **analysis** part, which breaks the source program into pieces and builds an intermediate representation, and a **synthesis** part, which constructs the target program from that intermediate representation. This is DB’s own high-level characterization of *every* compiler, regardless of source or target language.

2 Symbol Table and Error Handling: The Cross-Cutting Phases

[DB] Two components are not steps in the pipeline — they run *alongside every phase*, being read from and written to throughout the whole compilation.

- **Symbol table management.** A data structure recording every identifier in the program — variables, function names, class names — along with attributes: type, scope, memory location, and (for functions) parameter/return types. Lexical analysis creates entries; semantic analysis fills in type information; code generation uses it to compute memory addresses.
- **Error handling.** Each phase may encounter errors specific to its own level: lexical analysis catches malformed tokens, syntax analysis catches grammatically invalid programs, semantic analysis catches type mismatches and undeclared identifiers. A well-built compiler tries to *recover* from an error and keep going, so it can report several distinct errors from a single compilation run instead of stopping at the first one.



Extra Depth: What Real Symbol Tables Store

A production compiler's symbol table stores far more than DB's introductory description suggests: mangled names (for overload resolution in C++/Rust), calling-convention information, alignment and padding requirements, visibility/linkage (`static` vs. exported), inlining hints, and — in languages with generics/templates — a separate table per instantiation. Modern compilers often implement it not as one flat table but as a **chain of nested scopes** (a scope stack or scope tree), so that entering a block, function, or class pushes a new scope that shadows outer ones and is popped on exit. LLVM's `clang` keeps this information in its `Decl` hierarchy rather than a single table, reflecting how much richer real symbol-table designs are compared to the simple table DB introduces.

3 Grouping the Phases: Front-End, Middle-End, Back-End

[beyond DB] DB's own diagram groups phases into analysis/synthesis, but the vocabulary you will hear constantly in industry and in later lectures is **front-end** / **middle-end** / **back-end**. This is not just relabeling — it reflects a real engineering strategy.

Group	Contains / Responsibility
Front-end	Lexical, syntax, semantic analysis. <i>Language-specific</i> : knows the grammar and type rules of exactly one source language, and lowers it to a common IR.
Middle-end	Machine-independent optimization on the IR. <i>Language-agnostic</i> and <i>target-agnostic</i> : the same optimization passes (constant folding, dead-code elimination, inlining) apply no matter which source language or target CPU is involved.
Back-end	Code generation and machine-dependent optimization. <i>Target-specific</i> : knows the instruction set, register file, and calling convention of exactly one target architecture.

Why This Grouping Is the Whole Point

If front-ends only ever talk to a shared IR, and back-ends only ever talk to that same shared IR, then supporting a new *source language* only requires writing one new front-end — it automatically gets every optimization and every target the IR already supports. Symmetrically, supporting a new *target CPU* only requires writing one new back-end — it automatically works

for every source language already supported. Without this split, you would need to write a separate optimizer and code generator for every (language, target) *pair* — the classic “ M languages $\times N$ targets” problem, needing on the order of $M \times N$ components instead of $M + N$.

A Bit of History

This $M \times N$ argument predates LLVM by decades. In the 1960s, **T-diagrams** (popularized by F. L. Bauer and others) were a standard notation for exactly this problem: a T-shaped diagram showing a compiler written in language H , translating source language S into target language T . Compiler writers used T-diagrams to reason about *bootstrapping* (compiling a new compiler using an existing one) and about how many compilers you actually need to support M languages on N machines — precisely the front-end/back-end decoupling argument above, expressed sixty years earlier with pen-and-paper diagrams instead of shared IR software.

Extra Depth: The Front/Middle/Back Split in a Real Toolchain — LLVM

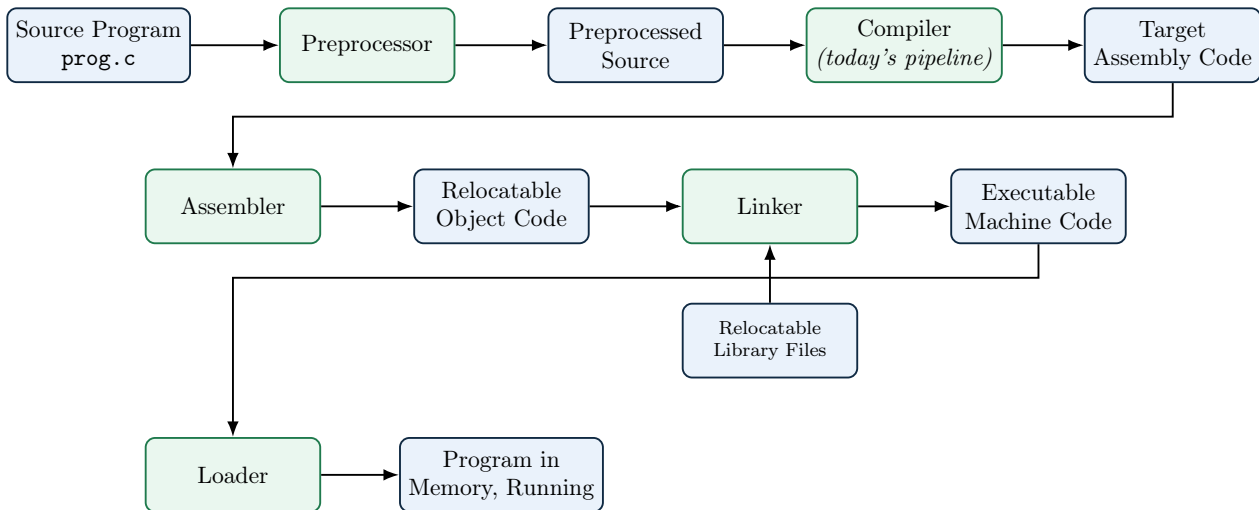
The clearest concrete instance of this architecture today is LLVM, and it is worth walking through once here since we will return to it in Lecture 25 for code generation proper.

- **Front-end (Clang, for C/C++/Objective-C):** lexes and parses source into a Clang-specific Abstract Syntax Tree (AST), performs semantic analysis and type checking on that AST, then *lowers* it into **LLVM IR** — a typed, SSA-based (Static Single Assignment) intermediate representation that is completely independent of C++ as a language. Rust (**rustc**), Swift, Julia, and Zig all have their own, completely different front-ends, but they all emit the *same* LLVM IR.
- **Middle-end (the LLVM optimizer, opt):** a pipeline of dozens of independent **passes** — e.g. **mem2reg** (promote stack variables to SSA registers), **instcombine** (peephole-simplify instruction sequences), **inline** (function inlining), **gvn** (global value numbering, subsuming common subexpression elimination), **loop-vectorize** — each of which reads LLVM IR and produces improved LLVM IR. None of these passes know or care whether the original source was C++, Rust, or Swift.
- **Back-end (llc, or the integrated code generator):** lowers optimized LLVM IR to a specific target’s machine code — x86-64, AArch64, RISC-V, or even WebAssembly, selected via a **-target** triple. Instruction selection, register allocation, and instruction scheduling all happen here, specific to that one target.

The practical payoff: when Apple Silicon (AArch64) needed a good C++ compiler, no one had to write a new C++ front-end — Clang’s C++ front-end and every middle-end optimization pass were already reusable; only a back-end for AArch64 was needed (and it mostly already existed, since LLVM had supported ARM for years).

4 The Compiler’s Neighbors: Preprocessor, Assembler, Linker & Loader

[DB] The “compiler” most people mean when they say `gcc file.c` is actually a **driver program** that invokes several distinct tools in sequence. DB §2.1–§2.8 places the compiler proper in the context of this larger **language-processing system**.



- **Preprocessor.** Handles macro expansion (`#define`), file inclusion (`#include`), and conditional compilation (`#ifdef`) *before* the compiler proper ever sees the code. It works on raw text, with no notion of the language's grammar.
- **Compiler (proper).** Everything in §1 of this lecture: source $\rightarrow$ target assembly (or directly to object code, in many modern compilers, skipping a textual assembly step).
- **Assembler.** Translates (textual) assembly instructions into **relocatable machine code** — binary instructions, but with addresses left as symbolic placeholders to be filled in later, since the assembler does not yet know the final memory layout.
- **Linker.** Resolves references *between* separately compiled object files and libraries — e.g. your `main.o` calling `printf`, defined in the C standard library, not in your own source. Produces one combined **executable** with all addresses resolved (or marked for the dynamic linker, see below).
- **Loader.** Takes the executable file, allocates memory for it, and places its instructions and data into memory so the operating system can begin executing it — the very last step before your program actually runs.

Exam Focus

Know precisely which of preprocessor/assembler/linker/loader are considered *part of* “the compiler” in the narrow sense (none of them — the compiler proper is only the six-phase pipeline from §1) versus which are separate programs in the surrounding toolchain that DB still expects you to know the role of. A common exam trap: “which phase resolves a call to a library function like `printf`?” — the answer is the **linker**, not the compiler.

Extra Depth: Seeing the Real Pipeline on Your Own Machine

You can watch this exact pipeline happen with GCC or Clang by stopping it at each stage:

- `gcc -E prog.c -o prog.i` — run *only* the preprocessor; inspect `prog.i` to see macros expanded and headers inlined.
- `gcc -S prog.i -o prog.s` — run the compiler proper; `prog.s` is human-readable target assembly.
- `gcc -c prog.s -o prog.o` — run the assembler; `prog.o` is a relocatable ELF (on Linux) object file. Inspect it with `objdump -d prog.o` or `nm prog.o`.
- `gcc prog.o -o prog` — run the linker, pulling in the C runtime and any libraries, producing the final executable.

Try this on a one-line "hello world" program: the jump in size and content between `prog.i` (hundreds of lines, from expanded standard-library headers) and `prog.c` (one line) is a genuinely eye-opening way to internalize what the preprocessor actually does.

Extra Depth: Static vs. Dynamic Linking, and What the Loader Really Does

DB's loader description is simplified; two refinements matter a great deal in practice:

- **Static linking** copies the code of every library function you use directly into your executable at link time. The result is self-contained but larger, and does not benefit when the OS wants to share one copy of, say, `libc`, across many running programs.
- **Dynamic linking** instead records, in the executable, only the *names* of needed shared libraries (`.so` on Linux, `.dll` on Windows, `.dylib` on macOS). A specialized program, the **dynamic linker/loader** (`ld.so` on Linux), runs as the executable starts, finds each shared library already resident in memory or maps it fresh, and **patches in** the real addresses of library functions — a process called **relocation** — before your `main` function ever runs. This is why dynamically linked executables are smaller on disk but have slightly higher startup latency and a runtime dependency on the right library versions being present (colloquially, "DLL hell").
- **Position-Independent Code (PIC)** is machine code generated so it can be loaded at *any* memory address, not a fixed one — essential for shared libraries (which may be mapped at different addresses in different processes) and for security features like ASLR (Address Space Layout Randomization). Generating PIC is a back-end code-generation decision, another example of how deep the "compiler" choices reach into topics normally considered OS territory.

Extra Depth: Build Systems Are Compiler Orchestration at Scale

Everything above describes compiling *one* file. Real projects have thousands. The tools that decide *which* files need recompiling and in what order are themselves built on the phase model above:

- **Make / Ninja** track file modification timestamps and a dependency graph to avoid recompiling unchanged files — essentially caching the output of the "compiler" node in the pipeline diagram above, keyed by source-file identity and timestamp.
- **ccache / sccache** go further: they hash the *preprocessed* source (post-preprocessor, pre-compiler) plus compiler flags, and cache the resulting object file, so an unchanged file compiled with unchanged flags never re-invokes the compiler proper at all — directly exploiting the preprocessor/compiler boundary from §4.
- **Bazel and other hermetic build systems** extend caching across an entire team or CI fleet: if any engineer, anywhere, has already built a given (source, flags, toolchain) combination, everyone else can fetch the cached object/executable instead of recompiling.
- **Link-Time Optimization (LTO / ThinLTO)** deliberately *breaks* the strict separate-compilation model: instead of each `.c` file being optimized in isolation before linking, LTO defers whole-program optimization to link time, when the optimizer can see every translation unit at once and, e.g., inline a function across what used to be file boundaries. This directly trades the modularity of the pipeline in §4 for better runtime performance.

Extra Depth: The Front-End's Second Career — IDEs and Language Servers

The front-end (lexer + parser + semantic analyzer) is valuable even when no target code is ever generated. This observation underlies the **Language Server Protocol (LSP)**: an editor like VS Code or Neovim talks to a background process (a “language server”) that re-runs a language’s front-end on every keystroke to provide autocomple, jump-to-definition, and real-time error squiggles — all without ever reaching code generation. `clangd` (built directly on Clang’s front-end), `rust-analyzer`, and `gopls` are all, in effect, “compilers with the back end removed and an editor bolted on instead.” This is a direct, practical payoff of the front-end/back-end separation from §3: a front-end built for compilation can be repurposed wholesale for tooling.

5 Self-Check Questions

Practice – Not Graded

1. Place the following in correct pipeline order: semantic analysis, linking, assembly, lexical analysis, loading, syntax analysis, preprocessing, code generation.
2. A program calls a function defined in a library it never `#includes` a declaration for, but the linker still resolves the call successfully because the symbol name matches at link time. Which phase would have caught this as an error if the declaration truly were missing, and why didn’t it?
3. Explain, in the M -languages/ N -targets argument, why a shared IR turns an $M \times N$ problem into an $M + N$ problem. What is the “shared IR” in LLVM’s case?
4. Why does dynamic linking trade smaller executables for slower startup, while static linking does the opposite?
5. `clangd` never performs code generation. Which phases of the pipeline does it still need to run, and why are those enough to power features like “jump to definition”?

Key Takeaways

- A compiler’s six classical phases, in order: lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization, code generation.
- These split into an **analysis** half (front end: builds an IR and symbol table) and a **synthesis** half (back end: builds the target program from that IR).
- The **symbol table** and **error handler** are not phases in the pipeline — they run alongside every phase.
- The **front-end/middle-end/back-end** split (language-specific $\rightarrow$ language-and-target-agnostic $\rightarrow$ target-specific) is what lets one shared IR (like LLVM IR) turn an $M \times N$ language/target problem into an $M + N$ problem.
- The compiler proper sits inside a larger toolchain: **preprocessor** (before) and **assembler, linker, loader** (after) are separate programs, not phases of the compiler itself, though DB expects you to know each one’s job.
- Front-ends have value beyond compilation: build caching, LTO, and language servers (`clangd`, `rust-analyzer`) all repurpose pieces of this pipeline for other ends.

References for This Lecture

DB Aho, Lam, Sethi, Ullman. *Compilers: Principles, Techniques, and Tools*, 2nd ed., §1.2, §2.1–§2.8.

PP Scott. *Programming Language Pragmatics*, §1.5, §1.6.

Lecture 3

Tokens, Patterns, and Lexemes; the Role of the Lexical Analyzer; Lexical Errors

CLO(s) covered:	CLO 1
Dragon Book [DB]:	§3.1 (The Role of the Lexical Analyzer), §3.3 (Specification of Tokens – terminology preview)
Other references:	[AP] Modern Compiler Implementation, Ch. 2

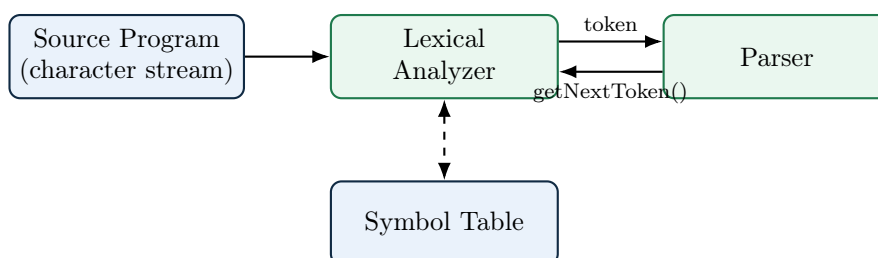
Lecture Roadmap

1. Where lexical analysis sits, and exactly how it talks to the parser.
2. Why lexer and parser are two separate phases instead of one.
3. The three core vocabulary words of this unit: **token**, **pattern**, **lexeme** — with several fully worked examples.
4. Token **attributes** and what actually gets passed on to the parser.
5. **Lexical errors**: what a lexer really can and cannot catch, and how it recovers.

This lecture builds the vocabulary and mental model. *How* patterns are written – regular expressions and finite automata – is Lectures 4–6.

1 Where Lexical Analysis Fits

[DB] The **lexical analyzer** (also called a **scanner** or **lexer**) is the first phase of a compiler. It reads the raw stream of characters making up the source program and groups them into a stream of **tokens** — meaningful chunks like keywords, identifiers, numbers, and operators — that the rest of the compiler works with instead of raw text.



How the Lexer and Parser Actually Talk to Each Other

In nearly every real compiler, the lexer does *not* tokenize the whole file up front and hand the parser a list. Instead, the parser drives the process: whenever the parser needs another token, it calls a function conventionally named `getNextToken()`. The lexer resumes scanning exactly where it left off, produces the next single token, and returns it. This “pull” model, sometimes called treating the lexer as a **coroutine** of the parser, avoids ever materializing the full token list in memory and lets the lexer and parser interleave their work one token at a time.

1.1 Why Not Just One Phase?

[DB] DB gives three concrete engineering reasons for splitting lexical analysis out as its own phase rather than folding it into the parser:

1. **Simplicity of design.** Separating out low-level tasks (recognizing keywords, numbers, string literals) lets the parser work purely with grammatical structure, using tokens as its atomic symbols, instead of also worrying about individual characters.
2. **Improved efficiency.** Specialized, table-driven techniques for recognizing tokens (built from finite automata, Lectures 5–6) are considerably faster than the general-purpose parsing techniques (Lectures 8 onward) would be if forced to operate character-by-character.
3. **Enhanced portability.** Input-device and operating-system-specific quirks — end-of-line conventions (`\n` vs. `\r\n`), character encodings, special end-of-file characters — are confined entirely to the lexer. The parser, and everything after it, is completely insulated from these differences.

Exam Focus

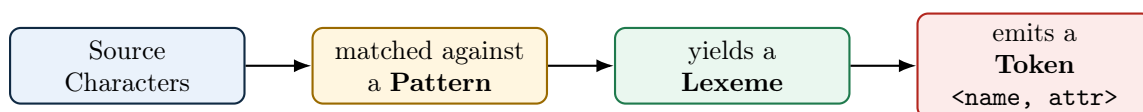
This “three reasons” list (simplicity, efficiency, portability) is a favorite short-answer question. Be able to name and briefly justify all three, not just one.

2 Tokens, Patterns, and Lexemes

[DB] These three terms are used precisely and are easy to blur together. Getting the distinction exactly right is one of the most exam-tested ideas in this entire unit.

Definition: Token, Pattern, Lexeme

- A **token** is a pair `<token-name, attribute-value>` representing a *category* of lexical unit, e.g. ID, NUM, RELOP. The token-name is an abstract symbol used by the parser.
- A **pattern** is the rule that describes the *set of possible lexemes* that can produce a given token – for identifiers, informally, “a letter followed by any number of letters and digits.” Lectures 4–6 make patterns precise using regular expressions.
- A **lexeme** is the actual sequence of characters in the source program that *matches* a pattern and is classified as an instance of some token.



Example: DB-Style Token/Pattern/Lexeme Table

Token	Informal Description of Pattern	Sample Lexemes
WHILE	the exact keyword <code>while</code>	<code>while</code>
ID	a letter, followed by any number of letters or digits	<code>i</code> , <code>sum</code> , <code>score2</code>
NUM	any numeric constant	<code>0</code> , <code>42</code> , <code>3.14</code>
RELOP	<code><</code> , <code><=</code> , <code>=</code> , <code><></code> , <code>></code> , or <code>>=</code>	<code><=</code> , <code><></code>
LITERAL	any characters between a pair of double quotes, except a double quote itself	<code>"Total = "</code>

2.1 Worked Example 1: Tokenizing a Function Call

Consider the following line of C-like source code:

```
printf ( "Total = %d\n" , score ) ;
```

identifier
 punctuation/operator
 string literal

Lexeme	Token Name	Attribute-Value (passed to parser)
<code>printf</code>	ID	pointer to symbol-table entry for <code>printf</code>
<code>(</code>	LPAREN	none needed
<code>"Total = %d\n"</code>	LITERAL	pointer to the string's stored value
<code>,</code>	COMMA	none needed
<code>score</code>	ID	pointer to symbol-table entry for <code>score</code>
<code>)</code>	RPAREN	none needed
<code>;</code>	SEMI	none needed

3 Attributes for Tokens

[DB] When more than one lexeme can match a token’s pattern (as with ID or NUM), the lexer must pass along enough extra information – the **attribute-value** – for later phases to know exactly *which* identifier or *which* number was matched. For identifiers, this is essentially always a pointer into the symbol table.

Example: Worked Example 2: An Assignment Statement

Source line: E = M * C ** 2

Assume `**` is this language's exponentiation operator. The lexer's output, as a stream of `<token-name, attribute-value>` pairs, is:

```
<ID, ptr to symtab entry for E>
<ASSIGN_OP, - >
<ID, ptr to symtab entry for M>
<MULT_OP, - >
<ID, ptr to symtab entry for C>
<EXP_OP, - >
<NUM, integer value 2>
```

Notice: ID tokens all carry a symbol-table pointer as their attribute so that later phases can distinguish E from M from C; the operator tokens need no attribute at all, since the token-name alone (ASSIGN_OP vs. MULT_OP) already says everything the parser needs to know.

Exam Focus

Given a short program line, be able to produce the exact stream of **<token-name, attribute-value>** pairs, in order, the way both worked examples above do. This is the single most common long-answer question style for this unit.

3.1 Worked Example 3: A Complete Statement Block

```
int sum = 0;
while (i <= 10) {
    sum = sum + i;
    i = i + 1;
}
```

Lexeme	Token	Attribute	Lexeme	Token	Attribute
int	TYPE	–	sum	ID	ptr(sum)
sum	ID	ptr(sum)	=	ASSIGN	–
=	ASSIGN	–	sum	ID	ptr(sum)
0	NUM	0	+	PLUS	–
;	SEMI	–	i	ID	ptr(i)
while	WHILE	–	;	SEMI	–
(	LPAREN	–	i	ID	ptr(i)
i	ID	ptr(i)	=	ASSIGN	–
<=	RELOP	LE	i	ID	ptr(i)
10	NUM	10	+	PLUS	–
)	RPAREN	–	1	NUM	1
{	LBACE	–	;	SEMI	–
			}	RBRACE	–

Two details worth noticing in this longer example:

- `sum` and `i` each get *their own* symbol-table pointer the first time they’re seen, and the *same* pointer every time they reappear – the symbol table guarantees one entry per distinct identifier, no matter how many times it’s used.
- Whitespace (the spaces and newlines between tokens) and indentation produce *no* tokens at all – they are consumed and discarded by the lexer, along with any comments, before the parser ever sees anything.

The Lexer’s “Housekeeping” Duties

Beyond producing tokens, DB notes the lexer is also the natural place to: strip out whitespace and comments (neither produces a token); and track line numbers (and often column numbers) so that later phases can report errors with accurate source locations, e.g. “undeclared variable ‘x’ at line 42.”

4 Lexical Errors

[DB] It is surprisingly rare for a lexer to be able to detect an error entirely on its own, *by itself, in isolation* – because almost any short character sequence is a *prefix* of some valid lexeme for some token.

Example: The Classic Illustration

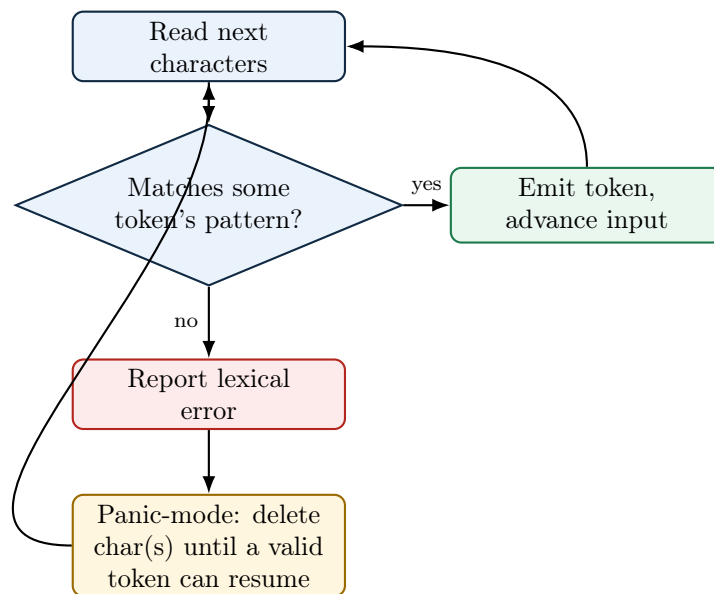
Consider the fragment `fi (a == f(x))` in a C-like language, where the programmer *meant* to type the keyword `if`. The lexer cannot know this. `fi` is a perfectly legal lexeme for token ID (“a letter followed by letters/digits”), so the lexer happily emits `<ID, ptr(fi)>`. The mistake will only be caught later – by the *parser*, if `fi` appears somewhere no identifier is grammatically valid, or by *semantic analysis*, if `fi` is used as if it were a function without ever having been declared.

Genuine lexical errors are ones where *no pattern for any token* can match the remaining input at all, or where a token’s own internal structure is malformed:

- An illegal character that starts no valid token in the language, e.g. a stray # or @ in a language that assigns those characters no meaning.
- An unterminated string literal: opening quote seen, but the line (or file) ends before a matching closing quote.
- A malformed numeric literal, e.g. 3.14.15 (two decimal points), or 2r immediately followed by a letter with no valid meaning in that position.
- An unterminated comment: a /* seen with no matching */ anywhere before end of file.

4.1 Error Recovery Strategies

[DB] A production-quality lexer does not simply stop at the first error — it tries to recover and keep going, so the programmer can see *several* distinct errors from one compilation attempt rather than fixing them one at a time.



[DB] DB lists several concrete local-correction strategies a recovering lexer can attempt, beyond plain panic mode:

- **Deleting** an extraneous character.
- **Inserting** a missing character.
- **Replacing** one incorrect character with a correct one.
- **Transposing** two adjacent characters.

A Bit of History

DB points toward a more principled version of this idea: choosing whichever single correction requires the *fewest* character edits – insertions, deletions, substitutions – to turn the bad input into something valid. This is exactly **edit distance** (Levenshtein distance), the same string-similarity measure used today in spell-checkers and DNA sequence alignment. Aho and Peterson described a minimum-distance error-correcting parser as early as 1972; the idea that “the best fix is the cheapest fix” turns out to generalize far beyond lexical analysis, into syntax error recovery and even runtime auto-correct systems.

Exam Focus

Be able to explain, with a concrete example like `fi (a == f(x))`, *why* a misspelled keyword is usually **not** a lexical error even though it looks like an obvious typo. Then be able to list at least two genuine, purely lexical errors (unterminated string, illegal character) and describe panic-mode recovery in one sentence.

5 Beyond the Dragon Book: How Real Lexers Handle Harder Cases

The Reserved-Word Trick

A naive lexer might need a separate hand-written check for every keyword (`if`, `while`, `return`, ...) before falling back to the generic identifier pattern. Production lexers instead use a much simpler trick: pre-load the symbol table with every reserved word *before* scanning begins, each marked with its own keyword token. The lexer then has only *one* pattern for anything “letter followed by letters/digits.” When it matches such a lexeme, it looks the lexeme up in the symbol table: if an entry already exists (because it’s a reserved word), the lexer returns *that* token (e.g. `WHILE`); if no entry exists, it’s a genuine user identifier, so the lexer creates a new symbol-table entry and returns `ID`. One pattern, one lookup, no special-casing in the automaton itself.

Maximal Munch – A Preview

When several patterns could all match a prefix of the remaining input, real lexers apply the **longest-match** (“maximal munch”) rule: always take the *longest* lexeme that matches *any* pattern. This is why `<=` is scanned as a single `RELOP` token and not as `<` followed by `=`, and why `i` in `int` is not prematurely cut off as an identifier before the lexer has looked far enough ahead to see the rest of the word. C’s `--` (decrement) versus `- -` (two unary minuses) and `->` (member access through a pointer) are further classic examples where getting the lookahead right matters. We formalize *why* this rule is needed, and how a scanner implements it efficiently, once we cover finite automata in Lectures 5–6.

Numeric Literals Are Far Messier Than “a Sequence of Digits”

DB’s `NUM` pattern is deliberately simplified. Real languages support numeric literal formats far beyond plain decimal integers, and each variant is a distinct lexical pattern the scanner must recognize:

Format	Example
Hexadecimal / octal / binary	<code>0x1F</code> , <code>0o17</code> , <code>0b1010</code>
Scientific notation	<code>6.022e23</code> , <code>1.5E-10</code>
Digit-group separators	<code>1_000_000</code> (Python, Java, Rust, C++14)
Type suffixes	<code>3.14f</code> (float), <code>100L</code> (long), <code>42u</code> (unsigned)
Complex/imaginary literals	<code>3+4j</code> (Python)

Each of these is its own regular pattern (or family of patterns) that a real scanner’s specification must include – one more reason lexer-generator tools (Lecture 7) are preferred over hand-written code once a language’s literal syntax grows this rich.

String Literals: Escapes, Raw Strings, and Interpolation

String-literal lexing hides real complexity:

- **Escape sequences** (`\n`, `\t`, `\\`, `\uXXXX`) must be recognized *inside* the literal's pattern, not treated as ending it.
- **Raw strings** (Python's `r"..."`, C#'s `@"..."`) deliberately *disable* escape processing – a single string-literal token category can therefore need two different sub-patterns.
- **String interpolation** / **f-strings** (Python's `f"Total = {score}"`, JavaScript template literals) are the hardest case: the lexer must recognize where an embedded *expression* begins inside a string and, in effect, briefly hand control back to a full expression lexer/parser before resuming the string literal. This breaks the assumption that lexical analysis is a single flat pass over characters, and is a genuinely active area of parser/lexer-design discussion in modern language implementations.

Not Every Language Is Purely Character-Driven: Indentation as Tokens

Python and Haskell use indentation, not braces, to delimit blocks. Their lexers cannot rely on simple per-character regular patterns alone – they maintain an explicit **indentation stack** and synthesize special `INDENT` and `DEDENT` pseudo-tokens whenever a logical line's indentation increases or decreases relative to the stack's top, in addition to the ordinary tokens for the line's actual content. This is a clean example of *why* DB's model of “patterns matched against a flat character stream” is the right *starting* point but not the end of the story for every real language.

Unicode Identifiers

DB's “a letter followed by letters and digits” quietly assumes ASCII. Modern language standards (Python 3, Swift, Julia, Rust as of recent editions) permit *any* Unicode character in the “Letter” category as part of an identifier – so accented Latin letters (`café`), non-Latin scripts (Greek, Arabic, Devanagari, Han characters), and even emoji are all valid identifiers in the languages that permit them (Swift famously allows an emoji as a variable name). Real lexer specifications reference the Unicode standard's character-category tables rather than a simple `[A-Za-z]` range.

Seeing Real Tokenizers at Work

You do not have to take any of this on faith – most language toolchains expose their tokenizer directly:

- Python: `python3 -m tokenize myfile.py` prints every token, its exact source position, and its type.
- Clang: `clang -Xclang -dump-tokens -fsyntax-only file.c` dumps the full token stream Clang's own lexer produced.
- Node.js/Acorn or Babel's own `-tokens` flags do the same for JavaScript.

Running one of these on a file with a deliberate typo – an unterminated string, an unexpected character – is the fastest way to see exactly what a real lexer reports as an error versus what it silently accepts and passes on to the parser.

6 Self-Check Questions

Practice – Not Graded

1. For the statement `total = total + rate * 60;`, write out the full stream of `<token-name, attribute-value>` pairs, following the style of Worked Examples 2 and 3.
2. Explain why `2x` (digit immediately followed by a letter) can be treated as a genuine lexical error in many languages, while `fi` cannot, even though both look like “obvious mistakes” to a human reader.
3. A lexer encounters an opening `"` with no matching closing quote before the end of the file. Describe two different, reasonable ways a lexer could recover from this and keep scanning the rest of the file.
4. Why must the longest-match (maximal munch) rule apply when scanning `i <= 10`, so that `<=` is not mistakenly split into `<` followed by `=`?
5. Explain, in your own words, why Python’s lexer cannot be built from character-level patterns alone the way a C lexer can.

Key Takeaways

- A **token** is an abstract `<name, attribute>` pair; a **pattern** is the rule describing which lexemes belong to a token; a **lexeme** is the actual matched source text. Keep these three cleanly separated.
- Lexer and parser are split into separate phases for **simplicity, efficiency, and portability** – know all three reasons.
- The lexer and parser typically interact via a pull-based `getNextToken()` call, not by materializing a full token list up front.
- Token attributes – almost always a symbol-table pointer for `ID` – are how later phases recover exactly *which* identifier or literal a token represents.
- Most apparent “typos” (like `fi` for `if`) are *not* lexical errors, because they are valid lexemes for some other token; genuine lexical errors are things no pattern can match at all (illegal characters, unterminated strings/comments, malformed numbers).
- Real-world lexers go well beyond DB’s simplified patterns: the reserved-word trick, maximal munch, rich numeric/string literal formats, indentation-based tokens, and Unicode identifiers are all standard in production compilers.

References for This Lecture

DB Aho, Lam, Sethi, Ullman. *Compilers: Principles, Techniques, and Tools*, 2nd ed., §3.1, §3.3.
 AP Appel. *Modern Compiler Implementation in C/Java/ML*, Ch. 2.

Lecture 4

Regular Languages vs. Non-Regular Languages; Regular Expressions and Regular Definitions

CLO(s) covered:	CLO 1
Dragon Book [DB]:	§3.3 (Specification of Tokens: Strings, Languages, Operations, Regular Expressions, Regular Definitions, Extensions)
Other references:	[PP] Programming Language Pragmatics §3.3

Lecture Roadmap

1. Strings, alphabets, and languages – the formal vocabulary underneath “pattern.”
2. Operations on languages: union, concatenation, and closure.
3. **Regular expressions:** the formal, inductive definition, plus many worked examples.
4. **Regular definitions:** naming sub-expressions, culminating in DB’s classic identifier and numeric-literal definitions.
5. Extensions (+, ?, character classes) – convenience, not new power.
6. **Regular vs. non-regular languages:** what a lexer’s patterns fundamentally cannot express, and why that is exactly the job handed to the parser.

Today makes “pattern” from Lecture 3 completely precise. Lectures 5–6 show *how* a machine matches a regular expression, via finite automata.

1 Strings, Alphabets, and Languages

[DB] Before regular expressions can be defined precisely, three primitive terms need to be pinned down exactly as DB uses them.

Definition: Alphabet, String, Language

- An **alphabet** Σ is any finite set of symbols, e.g. $\{0, 1\}$ or the set of ASCII characters.
- A **string** (or *word*) over Σ is a finite sequence of symbols drawn from Σ . The **empty string**, written ε , has length 0.
- A **language** over Σ is simply *any* set of strings over Σ — possibly infinite, possibly empty ($\emptyset$, the language with no strings at all, which is *not* the same as $\{\varepsilon\}$, the language containing only the empty string).

Exam Focus

$\emptyset$ and $\{\varepsilon\}$ are a classic confusion. $\emptyset$ contains *zero* strings; $\{\varepsilon\}$ contains *one* string (the empty one). Keep them distinct.

1.1 Operations on Languages

[DB] Given two languages L and M over the same alphabet, DB defines four operations that build new languages out of old ones. These four operations are the entire foundation regular expressions

are built from.

Operation	Notation	Definition
Union	$L \cup M$	$\{s \mid s \in L \text{ or } s \in M\}$
Concatenation	LM	$\{st \mid s \in L \text{ and } t \in M\}$ (paste a string from L directly onto a string from M)
Kleene closure	L^*	$\bigcup_{i=0}^{\infty} L^i$ – zero or more strings from L , concatenated (includes ε , from L^0)
Positive closure	L^+	$\bigcup_{i=1}^{\infty} L^i$ – one or more strings from L , concatenated (excludes ε unless $\varepsilon \in L$)

Example: Worked Example: Letters and Digits

Let $L = \{a, b, c, \dots, z, A, \dots, Z\}$ (the 52 letters) and $D = \{0, 1, \dots, 9\}$ (the 10 digits), each treated as a language of one-character strings.

- $L \cup D$ – every single letter *or* single digit: 62 strings, each of length 1.
- LD – every string formed by one letter followed by one digit, e.g. **a5**, **Z0** – exactly $52 \times 10 = 520$ strings, each of length 2.
- L^4 – every string of exactly 4 letters, e.g. **Test**, **abcd** – there are 52^4 such strings in total.
- L^* – every string of letters of *any* length, including ε – infinitely many strings.
- $L(L \cup D)^*$ – one letter, followed by zero or more letters-or-digits. This is *exactly* the language of identifiers in most C-like languages! We will turn this directly into a regular expression in §3.

2 Regular Expressions

[DB] A **regular expression** is a compact notation for describing a language. DB defines regular expressions *inductively*: a small set of base cases, plus rules for combining smaller regular expressions into bigger ones.

Definition: Regular Expressions (Inductive Definition)

Over alphabet Σ :

- **Base case 1:** ε is a regular expression; $L(\varepsilon) = \{\varepsilon\}$.
- **Base case 2:** for each symbol $a \in \Sigma$, a is a regular expression; $L(a) = \{a\}$.
- **Inductive case (union):** if r and s are regular expressions, so is $r \mid s$, with $L(r \mid s) = L(r) \cup L(s)$.
- **Inductive case (concatenation):** if r and s are regular expressions, so is rs , with $L(rs) = L(r)L(s)$.
- **Inductive case (closure):** if r is a regular expression, so is r^* , with $L(r^*) = (L(r))^*$.
- **Parenthesization:** if r is a regular expression, so is (r) , with $L((r)) = L(r)$ – used purely to control grouping/precedence.

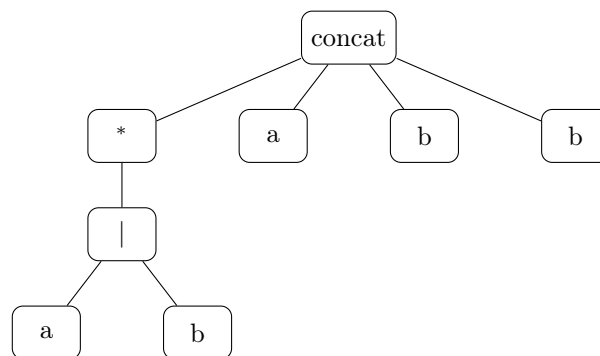
Operator Precedence

When parentheses are omitted, DB fixes a precedence, highest to lowest: **closure** ($*$) binds tightest, then **concatenation**, then **union** ($\mid$) binds loosest. So $ab \mid c$ means $(ab) \mid c$, *not* $a(b \mid c)$; and $a \mid bc^*$ means $a \mid (bc^*)$.

2.1 Worked Example: Building Up a Regular Expression

DB's running example: the language of all strings of a's and b's that *end in abb*. Let's build the regular expression piece by piece.

Sub-expression	What it describes
$a \mid b$	a single a <i>or</i> a single b
$(a \mid b)^*$	<i>any</i> string of a's and b's (including ϵ)
$(a \mid b)^*abb$	any string of a's and b's, followed by the literal suffix abb – i.e. exactly the strings that <i>end in abb</i>



*Syntax tree for $(a \mid b)^*abb$: the root concatenates four pieces — the starred union, then three literal symbols.*

Example: Trace: Does "babb" Match?

babb = **b** (matched by the $(a|b)^*$ part, one repetition) followed by **a**, **b**, **b** (the literal suffix). Yes, it matches. Does **"abab"** match? No – it does not end in the literal substring **abb** (it ends in **bab**).

Exam Focus

Given an informal language description (“strings that start with ...”, “strings with an even number of ...”, “strings that don’t contain ...”), be able to construct the regular expression, and vice versa: given a regular expression, describe its language in English and list 2–3 example strings it does and does not match. This two-way translation is the most common exam pattern for this topic.

2.2 More Worked Examples

Language (in English)	Regular Expression	Matches / Doesn't Match
Binary strings with an even number of 0's	$1^*(01^*01^*)^*$	Matches 110011; doesn't match 100
Strings over $\{a, b\}$ containing at least one a	$(a \mid b)^*a(a \mid b)^*$	Matches bba; doesn't match bbb
Strings over $\{a, b\}$ <i>not</i> containing the substring aa	$b^*(ab^+)^*a^?$	Matches ababb; doesn't match baab
C-style single-line comment	$//(\Sigma \setminus \{\text{newline}\})^*$	Matches <code>// TODO fix this;</code> doesn't match a two-line comment

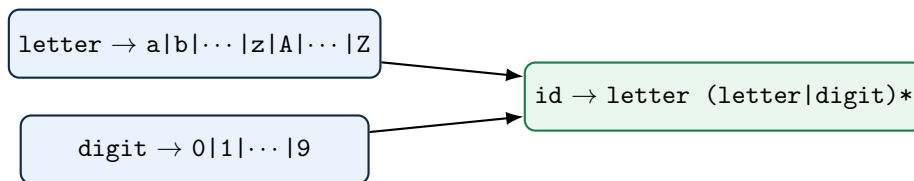
3 Regular Definitions

[DB] Writing out a complex regular expression as one giant unreadable formula is painful. A **regular definition** lets us name intermediate regular expressions and reuse those names in later definitions — exactly like defining helper variables in ordinary algebra.

Definition: Regular Definition

A regular definition is a sequence $d_1 \rightarrow r_1, d_2 \rightarrow r_2, \dots, d_n \rightarrow r_n$ where each d_i is a new name, and each r_i is a regular expression over $\Sigma \cup \{d_1, \dots, d_{i-1}\}$ — that is, r_i may use the alphabet *and* any name defined *before* it, but never itself or a later name (no recursive or forward references).

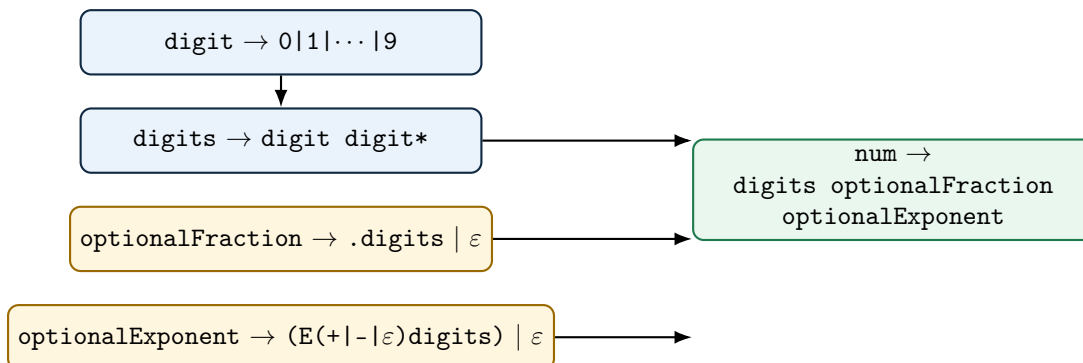
3.1 Worked Example: Identifiers



This is exactly $L(L \cup D)^*$ from the worked example in §1.1, now written as a named, reusable regular definition — and it is the pattern for token ID in almost every C-family language.

3.2 Worked Example: Unsigned Numbers (DB's Classic Definition)

Numbers like 5280, 0.01, 6.336E4, or 1.89E-4 all need to be matched by a single token pattern. DB builds this up from four named pieces:



Example: Tracing num Against 6.336E4

`digits` matches 6; `optionalFraction` matches .336 (the `.digits` branch); `optionalExponent` matches E4 (the ϵ -sign branch of $(E(+-|\epsilon)digits)$, since no `+-` appears). Concatenated: `6 + .336 + E4 = 6.336E4`. For a plain integer like 5280, both `optionalFraction` and `optionalExponent` take their ϵ branch and contribute nothing.

Exam Focus

Be able to reproduce this exact four-piece `num` definition from memory, and trace it against a given number (integer, decimal, or scientific-notation) the way the example above does. This is DB's single most frequently examined regular-definition example.

4 Extensions of Regular Expressions

[beyond DB] Modern regex notation (and DB §3.3.5) adds a few extra operators. Formally, none of these add any expressive power beyond plain union/concatenation/closure — they are **syntactic**

sugar, purely for convenience and readability.

Extension	Meaning	Equivalent to Core Regex
r^+	one or more repetitions	rr^*
$r^?$	zero or one occurrence (optional)	$r \mid \varepsilon$
$[abc]$	character class: any one of a , b , c	$a \mid b \mid c$
$[a-z]$	character range: any letter from a to z	$a \mid b \mid \dots \mid z$
$\neg a$	negated class: any symbol <i>except</i> a	union of all $x \in \Sigma \setminus \{a\}$

Exam Focus

Know that $r^+ \equiv rr^*$ and $r^? \equiv (r \mid \varepsilon)$ *exactly* — a common trick question asks you to rewrite an extended-notation regex using only the three core operators (union, concatenation, closure), which is precisely this substitution.

5 Regular vs. Non-Regular Languages

[DB] Not every language can be described by a regular expression. This matters enormously for compiler design: it is exactly *why* a single lexer pass is not enough, and why we need an entirely different, more powerful formalism (context-free grammars, Lecture 8) for parsing.

The Central Limitation: No Counting

A regular expression (equivalently, a finite automaton, Lectures 5–6) has no way to *count* an unbounded quantity and later check that count against something else. It can verify local, bounded patterns, but it cannot verify *global balance*. This single limitation explains every classic example of a non-regular language.

Language	Regular?	Why
Identifiers: letter followed by letters/digits	Yes	Purely local: check each character against a fixed small set of rules, no counting needed.
Fixed-format numeric literals	Yes	Same reason – bounded structure, no unbounded counting or matching required.
Balanced parentheses, e.g. $\{a^n b^n \mid n \geq 0\}$	No	Requires counting how many (were seen and demanding <i>exactly</i> that many) later — an unbounded count with no fixed upper limit.
Matching HTML/XML open and close tags	No	Same reason, at the level of tags instead of characters – and worse, tags can <i>nest</i> , requiring a stack, not just a counter.
Palindromes over $\{a, b\}$	No	Requires comparing the first half of the string against the reverse of the second half – again, unbounded memory of what came before.

Example: Why Finite Automata Can't Count: A Concrete Illustration

Suppose you tried to build a finite automaton for balanced parentheses up to nesting depth n . You would need at least one distinct state per depth level, $0, 1, 2, \dots, n$, just to remember “how many unmatched (have I seen so far.” Since parentheses in real programs can nest arbitrarily deep, no *fixed, finite* number of states suffices for *every* possible program – yet a finite automaton is required to have a fixed number of states, decided in advance. This is the intuitive heart of the **pumping lemma for regular languages** from your Automata Theory prerequisite: pump the middle of a long enough matched string and you break the balance, proving no finite automaton can recognize it.

Bridging Idea – Why This Matters for the Rest of the Course

- Regular expressions/finite automata are the right tool for tokens: identifiers, numbers, keywords, operators – all locally-checkable, bounded patterns.
- Nested and recursive structure – matched parentheses, balanced braces, nested expressions, `if/else` matching – is fundamentally *beyond* what regular expressions can express, no matter how cleverly you write them.
- This is precisely why Lecture 8 introduces **context-free grammars**: a strictly more powerful formalism (using a stack, conceptually) that *can* count and match nested structure, and precisely why the compiler needs *two* separate formalisms — one per phase — instead of one universal one.

6 Beyond the Dragon Book: Regex in Practice

Not All “Regex” Is Actually Regular

This is a genuinely important and often-missed subtlety. Tools like Perl, Python's `re`, and JavaScript's regex support **backreferences** – e.g. `(\w+) \1` matches any word immediately repeated, like `hello hello`. This is provably *not* a regular language in the formal CS-theory sense used in this lecture — recognizing it requires comparing two substrings for equality, which needs unbounded memory, just like the palindrome example above. Backreference-supporting engines use **backtracking search**, not finite automata, under the hood; they are more accurately called “backtracking pattern matchers” than true regular-expression engines. Meanwhile, lexer-generator tools like Flex (Lecture 7) implement only *true* regular expressions, compiled to genuine finite automata, precisely because lexical analysis needs the speed and predictability guarantees only real finite automata provide.

Catastrophic Backtracking (ReDoS)

Backtracking regex engines can, on adversarial input, take *exponential* time on a pattern that looks entirely innocent. The classic example is `(a+)+b` applied to a long string of `a`'s with no trailing `b`, like `"aaaaaaaaaaaaaaaaaaaaaaaaaaaaa!"`: the engine tries exponentially many ways of splitting the `a`'s between the inner and outer `+` before giving up. This is a real, named security vulnerability class — **ReDoS** (Regular expression Denial of Service) — and has caused real production outages (Cloudflare's July 2019 global outage was triggered by exactly this kind of catastrophic backtracking in a WAF rule). The illustrative growth pattern:

Input length (a's, no trailing b)	20	25	30	35
Illustrative matching steps (order of magnitude)	$\sim 10^5$	$\sim 10^6$	$\sim 10^8$	$\sim 10^{10}$

This is exactly why the DFA-based matching you will learn in Lectures 5–6 – which is *provably* linear in input length, with no possibility of this blow-up – is not just a theoretical nicety but the actual engineering reason production lexers never use a naive backtracking matcher.

Modern Guaranteed-Linear Engines

Google's **RE2** library and Rust's **regex** crate deliberately give up backreferences and a handful of other backtracking-only features specifically so that they can guarantee linear-time matching via automata-based execution, immune to ReDoS by construction. This is the industrial embodiment of the regular-vs-non-regular distinction from §5: by refusing to support genuinely non-regular features, these libraries buy back the performance guarantee that true finite automata provide.

Brzowski Derivatives – An Alternative to Automata

Lectures 5–6 build a matcher for a regex by first converting it to a finite automaton (via Thompson's construction). There is a completely different, more recent technique worth knowing about: the **derivative** of a regular expression r with respect to a symbol a , written $\partial_a(r)$, is a new regular expression describing “what still needs to match after consuming an a .” Repeatedly taking derivatives, one input symbol at a time, and checking whether the final derivative accepts ε , matches a string *without ever building an explicit automaton at all*. Janusz Brzowski introduced this in 1964; it fell out of fashion for decades, then saw a resurgence starting around 2010 in functional-programming circles (Scala's and Racket's parser-combinator libraries use derivative-based matching), because derivatives compose beautifully with recursive data types in a way that hand-built automata do not.

Real-World Regex Flavors Differ

The regex you type into **grep**, Python, or a Flex specification are not identical languages – they share the core ideas from this lecture but differ in surface syntax and sometimes in power:

Flavor	Notable differences
POSIX ERE (grep -E)	Closest to DB's formal core; no backreferences; character classes like <code>[:digit:]</code> .
PCRE / Python re / JavaScript	Add backreferences, lookahead/lookbehind assertions, non-greedy <code>*?</code> , named groups – genuinely more expressive than regular languages.
Flex / lex specifications	Core regular expressions plus start conditions and direct integration with symbol-table actions – built to compile to a real DFA (Lecture 7).

Knowing which flavor you're targeting matters: a pattern copied from a Python script into a Flex `.l` file may silently mean something different, or fail to compile at all.

7 Self-Check Questions

Practice – Not Graded

1. Write a regular expression for all strings over $\{0,1\}$ that *start* with 1. Then write one for strings that *do not contain* the substring 00.
2. Using DB's `num` regular definition from §3.2, trace the match for the literal `1.89E-4`, showing what each of `digits`, `optionalFraction`, and `optionalExponent` matches.
3. Rewrite r^+ and $r^?$ using only union, concatenation, and Kleene closure (no extension operators allowed).
4. Explain, in your own words and without using the word “count,” why balanced parentheses of arbitrary depth cannot be recognized by any regular expression.
5. A teammate writes the regex `(a|aa)+b` and complains their program hangs on long inputs with no trailing `b`. What phenomenon is this, and why does it happen with a backtracking engine but not with a DFA-based one?

Key Takeaways

- A language is just a set of strings; regular expressions are one *notation* for describing certain languages, built inductively from ε , single symbols, union, concatenation, and Kleene closure.
- Precedence, highest to lowest: closure ($*$), then concatenation, then union ($|$).
- **Regular definitions** let you name and reuse sub-expressions; DB's `id` and `num` definitions are the two canonical worked examples to know cold.
- Extension operators ($+$, $?$, character classes) add convenience, not expressive power – each has an exact equivalent using only the three core operators.
- Regular expressions fundamentally **cannot count** an unbounded quantity, which is exactly why balanced parentheses, nested tags, and palindromes are *not* regular – and exactly why the compiler needs a second, more powerful formalism (context-free grammars) for parsing.
- Not everything called “regex” in practice is regular in the formal sense – backreference-supporting engines use backtracking and can suffer catastrophic (ReDoS) performance; true finite-automaton-based matching, which we build next, is immune to this by construction.

References for This Lecture

DB Aho, Lam, Sethi, Ullman. *Compilers: Principles, Techniques, and Tools*, 2nd ed., §3.3.
 PP Scott. *Programming Language Pragmatics*, §3.3.